Computer Vision for Robotics

A Complete Guide — From First Convolution to Vision-Language Models

Prepared for Sunkeerth

September 2026

Contents

1 Chapter 1 — How Computers See **5**
- 1.1 Pixels and colour spaces . . . 5
- 1.2 The convolution operation . . . 5
- 1.3 Key convolution parameters . . . 5
- 1.4 Why stack many layers? . . . 6
- 1.5 The core building blocks of a CNN . . . 7
- 1.6 A minimal CNN in PyTorch . . . 7
- 1.7 Receptive field — how much of the image a neuron 'sees' . . . 8
- 1.8 Common pitfalls when building your first CNN . . . 8
- 1.9 Case study: a from-scratch colour-marker detector . . . 8
- 1.10 Convolution as matrix multiplication (im2col) . . . 9

2 Chapter 2 — The Evolution of CNN Architectures **9**
- 2.1 The lineage, level by level . . . 10
- 2.2 Architecture family comparison . . . 11
- 2.3 Transfer learning: standing on these shoulders . . . 12
- 2.4 Case study: picking a backbone for a Jetson-class robot . . . 12
- 2.5 Common pitfalls when choosing an architecture . . . 12
- 2.6 Visualising what a CNN has learned: Grad-CAM . . . 12

3 Chapter 3 — Training Deep Vision Models **13**
- 3.1 Loss functions . . . 13
- 3.2 Optimizers . . . 14
- 3.3 Data augmentation . . . 14
- 3.4 Regularisation — preventing overfitting . . . 15
- 3.5 Learning-rate schedules . . . 15
- 3.6 A complete (short) training loop . . . 15
- 3.7 Debugging a training run . . . 16
- 3.8 Case study: overfitting on a small Gazebo-captured dataset . . . 16
- 3.9 Mixed-precision training . . . 16

4 Chapter 4 — Object Detection **17**
- 4.1 Two-stage vs one-stage detectors . . . 17
- 4.2 Anchors, IoU, and non-max suppression . . . 18

4.3 The YOLO family, architecturally . . . 18
4.4 Speed vs accuracy: choosing a detector . . . 18
4.5 A minimal YOLO inference call (Python) . . . 19
4.6 Fine-tuning a detector on your own classes . . . 19
4.7 Common pitfalls in detection . . . 20
4.8 Case study: tuning confidence threshold for a moving robot . . . 20
4.9 Worked example: computing IoU by hand . . . 20

5 Chapter 5 — Segmentation & Pose Estimation 21
5.1 Three flavours of segmentation . . . 21
5.2 How Mask R-CNN works . . . 22
5.3 Foundation segmentation models . . . 22
5.4 Keypoint and pose estimation . . . 22
5.5 From 2D mask to 3D grasp point . . . 22
5.6 Segmentation vs pose vs detection — when to use which . . . 23
5.7 Common pitfalls in segmentation and pose . . . 23
5.8 Case study: grasp point on a coffee mug . . . 23
5.9 A note on point cloud processing . . . 23

6 Chapter 6 — Evaluating Vision Models 24
6.1 Classification metrics . . . 24
6.2 The confusion matrix . . . 25
6.3 Detection metrics: IoU and mAP . . . 25
6.4 Segmentation metrics . . . 25
6.5 Latency and throughput — the robotics-specific metrics . . . 25
6.6 A simple benchmarking snippet . . . 26
6.7 Common pitfalls in evaluation . . . 26
6.8 Case study: computing mAP for your own dataset . . . 26
6.9 Cross-validation for small datasets . . . 27

7 Chapter 7 — Attention and Transformers 27
7.1 Self-attention, mechanically . . . 28
7.2 Multi-head attention . . . 28
7.3 Vision Transformer (ViT) . . . 28
7.4 CNN vs Transformer for vision . . . 29
7.5 Hybrid and efficient variants . . . 29
7.6 A minimal self-attention block . . . 29
7.7 Positional encoding, in a bit more depth . . . 30
7.8 Common pitfalls with Transformer-based vision models . . . 30
7.9 Relative and rotary positional encodings . . . 30

8 Chapter 8 — Vision-Language Models (VLMs) 31
8.1 CLIP — the foundation of modern VLMs . . . 31
8.2 From CLIP to full VLMs . . . 32
8.3 Zero-shot classification with CLIP . . . 32
8.4 Why VLMs matter for robotics . . . 32
8.5 Open-vocabulary detection: bridging VLMs and Chapter 4 . . . 32
8.6 Prompt engineering for VLMs in robotics . . . 33

8.7 Case study: a VLM-guided pick instruction . . . 33
8.8 Common VLM grounding failure modes . . . 33

9 Chapter 9 — Choosing the Right Model 34
9.1 Decision checklist . . . 34
9.2 A practical recommendation table . . . 34
9.3 A worked example: choosing for a bipedal humanoid . . . 35
9.4 A note on cost of iteration . . . 35
9.5 A cost-of-ownership perspective . . . 35
9.6 Common mistakes when picking a model . . . 35
9.7 Ensembling: combining multiple models . . . 36

10 Chapter 10 — Computer Vision in Robotics 37
10.1 The perception-to-action pipeline . . . 37
10.2 Sensor fusion . . . 37
10.3 Visual SLAM . . . 38
10.4 Where each CV component sits in a robot stack . . . 38
10.5 The ROS 2 perception message flow . . . 39
10.6 Camera calibration: intrinsics and extrinsics . . . 39
10.7 Common pitfalls in sensor fusion . . . 39
10.8 Sharing perception across multiple robots . . . 40

11 Chapter 11 — Connecting Vision to Reinforcement & Imitation Learning 40
11.1 Vision as the observation for an RL agent . . . 41
11.2 Reinforcement learning vs. imitation learning . . . 41
11.3 Common RL algorithms used with visual observations . . . 42
11.4 The sim-to-real workflow (what you'll actually do) . . . 42
11.5 Reward shaping from vision . . . 42
11.6 A minimal Gymnasium-style wrapper around a Gazebo camera observation . . . 43
11.7 Common pitfalls in vision-based RL . . . 43
11.8 Case study: reward shaping for an approach task . . . 43
11.9 Curriculum learning . . . 44

12 Chapter 12 — Mini-Project Blueprint: CV-Enabled Robot in ROS 2 + Gazebo 44
12.1 Project architecture . . . 45
12.2 Step-by-step build order . . . 45
12.3 Minimal ROS 2 perception node (Python) . . . 45
12.4 Package layout . . . 46
12.5 From-scratch option . . . 46
12.6 Suggested milestones . . . 46
12.7 Testing and debugging the pipeline . . . 47
12.8 Extending the project further . . . 47
12.9 Continuous integration for a ROS 2 perception package . . . 47

13 Chapter 13 — Deployment & Edge Optimisation 48
13.1 Quantisation . . . 48
13.2 Pruning and distillation . . . 48
13.3 ONNX and TensorRT . . . 49

13.4 Benchmarking on Jetson-class hardware . . . 49
13.5 A practical deployment checklist . . . 49
13.6 Common pitfalls in deployment . . . 50
13.7 Case study: shrinking a segmentation model for Jetson Nano . . . 50
13.8 Model versioning and rollback . . . 50

14 Chapter 14 — Platforms & Tooling Comparison 51
14.1 Simulators . . . 51
14.2 Middleware & languages . . . 51
14.3 Visualisation . . . 52
14.4 Dataset and labelling tools . . . 52
14.5 Suggested toolchain for your project . . . 52
14.6 Migration and version considerations . . . 52
14.7 Final checklist before you start building . . . 52
14.8 Cloud vs edge inference . . . 53

1 Chapter 1 — How Computers See

By the end of this chapter you will be able to:

- Explain how a digital image is represented numerically, and why colour space choice matters.
- Describe the convolution operation and compute what a single filter pass does to an image patch.
- Name the five core CNN building blocks and the role each plays.
- Build and train a minimal CNN from scratch in PyTorch.

A digital image is nothing but a grid of numbers. A greyscale image is a 2D matrix of pixel intensities (0–255); a colour image is three such matrices stacked — Red, Green, Blue. Everything computer vision does, from the earliest edge detectors to today's vision-language models, is built on manipulating this grid of numbers to extract meaning.

1.1 Pixels and colour spaces

The most common representation is RGB, where each pixel is a triple of intensities. But RGB is not always the most useful space to work in:

- **Grayscale** — a single channel; used when colour carries no information (e.g. some LiDAR-projected intensity images).
- **HSV (Hue, Saturation, Value)** — separates colour identity from brightness; useful for colour-based segmentation that must be robust to lighting changes, e.g. detecting a specific coloured marker on a robot arm.
- **Depth maps** — not colour at all, but a per-pixel distance value from an RGB-D or stereo camera; essential for grasping and obstacle avoidance.
- **Normalisation** — in practice, pixel values are rescaled to [0,1] or standardised (subtract mean, divide by standard deviation) before being fed to a network, because neural networks train far more reliably on small, zero-centred inputs.

1.2 The convolution operation

A convolutional filter (or kernel) is a small matrix — typically 3×3 or 5×5 — of learnable weights. It slides across the image, and at every position it computes a dot product between the filter weights and the pixels underneath it. This produces one number per position, and the collection of these numbers across the whole image is called a feature map.

Analogy (The flashlight in a dark room).

Imagine scanning a dark wall with a small flashlight, one patch at a time. Each patch you illuminate tells you a little bit about the local texture — is it smooth, is there an edge, is there a corner? A convolution filter does exactly this: it never looks at the whole image at once, only a small local patch, and it reports back one number that summarises what it 'saw' in that patch, such as 'yes, there's a vertical edge here.'

1.3 Key convolution parameters

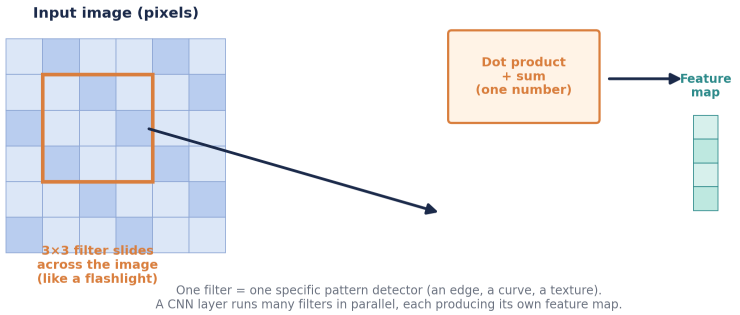


Figure 1: A single convolution filter scanning an image to produce a feature map.

Parameter	What it controls	Typical robotics-relevant effect
Kernel size	Size of the local patch examined (3×3, 5×5, 7×7)	Larger kernels see more context per layer but cost more compute — important on embedded GPUs
Stride	How far the filter moves between applications	Stride 2 downsamples the feature map, reducing compute for real-time perception
Padding	Whether the border is padded with zeros	'Same' padding keeps feature map size constant — useful when precise pixel alignment matters (e.g. for pose keypoints)
Dilation	Spacing between kernel elements	Expands the receptive field without extra parameters — useful for scene segmentation where wide context matters
Channels (depth)	Number of filters per layer, i.e. number of feature maps produced	More channels = more pattern types detected, at higher memory and compute cost

1.4 Why stack many layers?

Layer 1 filters learn simple things: edges, colour blobs, corners.

Layer 2–3 combine those into textures and simple shapes (circles, stripes).

Deeper layers combine shapes into parts (an eye, a wheel, a wrench-head) and finally whole objects.

This is called hierarchical feature learning — the network builds a vocabulary of increasingly abstract visual concepts, layer by layer, entirely from data, without anyone hand-coding what an 'edge' or a 'wheel' looks like.

Analogy (Detectives passing notes up a chain).

Think of the network as a chain of detectives. The first detective only notices raw clues — smudges, scratches, colour changes (edges). She passes her notes to the second detective, who combines several smudges into 'ah, this looks like a corner or a curve.' The third combines corners into 'this looks like a wheel-shape.' By the last detective in the chain, the notes say 'I am 97% confident this is a car.' No single detective understands the whole picture — understanding emerges from the chain.

1.5 The core building blocks of a CNN

Layer type	What it does	Why it matters
Convolution	Slides filters over the input to produce feature maps	Detects local patterns; weights are shared, so it is efficient and translation-invariant
Activation (ReLU)	Zeroes out negative values, non-linear	Without it, stacking layers would collapse into one linear function
Pooling (Max/Avg)	Downsamples feature maps (e.g. 2×2 → 1)	Reduces computation and adds robustness to small shifts
Batch Normalisation	Normalises activations within a batch	Stabilises and speeds up training
Fully Connected	Flattens features into a vector, maps to output classes	Turns spatial features into the final decision

1.6 A minimal CNN in PyTorch

To ground the theory, here is the smallest CNN that can classify an image — the kind of thing you would build from scratch before reaching for a pretrained model:

```
import torch.nn as nn
class TinyCNN(nn.Module):
def __init__(self, num_classes=10):
super().__init__()
self.features = nn.Sequential(
nn.Conv2d(3, 16, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2), # 32x32 -> 16x16
nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2), # 16x16 -> 8x8
)
self.classifier = nn.Sequential(
nn.Flatten(),
nn.Linear(32 * 8 * 8, 128), nn.ReLU(),
nn.Linear(128, num_classes),
)
def forward(self, x):
x = self.features(x)
return self.classifier(x)
```

This 4-layer network already illustrates the whole recipe: convolve → activate → pool → repeat → flatten → classify. Everything in the rest of this book is an elaboration of this same recipe at greater depth and sophistication.

1.7 Receptive field — how much of the image a neuron 'sees'

A single neuron in the first convolution layer only sees a 3×3 patch of the raw image. But a neuron in the second layer, built from those first-layer outputs, indirectly sees a larger area — its receptive field grows with depth. This is precisely why deep networks can eventually recognise whole objects: by the final layers, each neuron's receptive field can cover the entire image.

Robotics takeaway. When you design a perception network for a small robot camera, receptive field size should roughly match the size of the objects you care about relative to the frame. Too small a receptive field at the final layer, and the network can only ever describe textures, never whole objects.

1.8 Common pitfalls when building your first CNN

Symptom	Likely cause	Fix
Loss stays flat, never decreases	Learning rate too low, or inputs not normalised	Normalise inputs to [0,1] or standardise; try a larger learning rate
Loss becomes NaN	Learning rate too high, or unstable activation	Lower the learning rate; add batch normalisation
Training accuracy high, validation accuracy low	Overfitting — too little data or too large a model	Add augmentation/dropout, or use a smaller network / transfer learning
Output size doesn't match expectation	Miscounted the effect of pooling/stride on spatial dimensions	Trace the H×W through each layer by hand, or print shapes during a forward pass

1.9 Case study: a from-scratch colour-marker detector

A good first project is a CNN that classifies whether a small camera crop contains a specific coloured marker (e.g. a red calibration dot on a robot arm) versus background. This is small enough to label a few hundred crops by hand, train the TinyCNN from Section 1.6 in minutes on a laptop CPU, and gives a complete, understandable pipeline before introducing any pretrained model.

1. Capture ˜300 image crops from the Gazebo camera topic, half containing the marker, half not.

2. Resize all crops to a fixed size (e.g. 32×32) and normalise pixel values.

3. Train TinyCNN for ˜20 epochs with the loop from Chapter 3; watch the validation accuracy, not just training accuracy.

4. Slide the trained classifier over a full camera frame in a coarse grid to turn it into a crude detector — a hands-on preview of why Chapter 4's detectors exist: this brute-force sliding approach is exactly what YOLO-style single-pass detection was invented to avoid.

1.10 Convolution as matrix multiplication (im2col)

On real hardware, convolution is rarely computed as a literal sliding-window loop — that would be far too slow. Instead, most frameworks use an approach called im2col: overlapping image patches are unrolled into columns of a large matrix, and the convolution becomes a single big matrix multiplication between this matrix and the filter weights. This reframing is what lets convolutions run efficiently on GPUs, which are highly optimised for matrix multiplication. Understanding this is useful when reasoning about why certain kernel sizes, strides, or channel counts are dramatically faster or slower in practice than a naive parameter count would suggest — the underlying matrix shapes, and how well they fit the GPU's memory hierarchy, matter as much as the raw operation count.

Chapter Summary

- Images are grids of numbers; convolution filters slide over them to detect local patterns.
- Stacking layers builds a hierarchy from edges to textures to whole objects.
- The five core building blocks are convolution, activation, pooling, batch norm, and fully connected layers.
- Receptive field size determines how much spatial context a neuron can 'see'.

Review Questions

1. Why does a convolution filter only look at a small local patch instead of the whole image at once?
2. What is the difference between the receptive field of a neuron in layer 1 versus layer 5 of a CNN?
3. Name three of the five core CNN building blocks and explain what each one does.
4. Why is a non-linear activation function (like ReLU) necessary between convolution layers?
5. If your validation accuracy is much lower than training accuracy, what is happening, and what are two ways to address it?
6. In the 'flashlight' analogy, what does the size of the flashlight beam correspond to in a real CNN?
7. What does the im2col trick let a GPU compute convolution as, and why does that matter for speed?
8. Why is HSV sometimes preferred over RGB for colour-based segmentation that must be robust to lighting changes?

2 Chapter 2 — The Evolution of CNN Architectures

By the end of this chapter you will be able to:

- Trace the major CNN architecture milestones from LeNet to the Vision Transformer.
- Explain why residual (skip) connections made very deep networks trainable.
- Compare architectures by depth, parameter count, and the innovation each introduced.

- Apply transfer learning to fine-tune a pretrained backbone on a new, small dataset.

Once the basic recipe existed, the field's history is really the history of answering one question repeatedly: how do we make the network deeper and more capable without it becoming untrainable or impossibly slow?

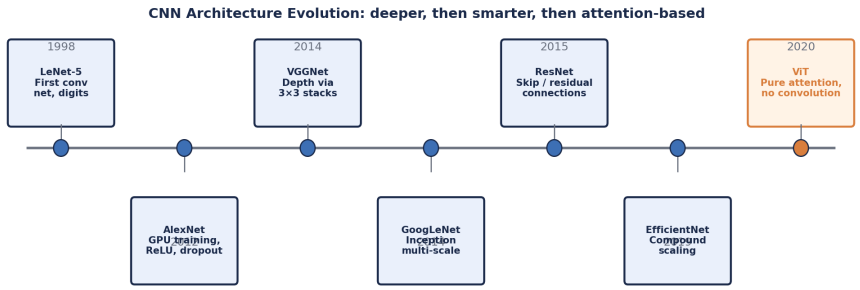


Figure 2: CNN architecture evolution from 1998 to the arrival of attention-based vision models.

2.1 The lineage, level by level

LeNet-5 (1998)

The prototype. Two convolution layers, two pooling layers, used to read handwritten digits. Small, but it proved the concept end-to-end.

AlexNet (2012)

Scaled the same idea up using GPUs, ReLU activations, and dropout regularisation, and won ImageNet by a huge margin. This is the moment deep learning became the default approach to vision.

VGGNet (2014)

Showed that stacking many small 3×3 filters (instead of a few large ones) gives the same receptive field with fewer parameters and more non-linearity. Simple, uniform, still used today as a feature extractor baseline.

GoogLeNet / Inception (2014)

Introduced the 'Inception module': run several filter sizes (1×1, 3×3, 5×5) in parallel on the same input and concatenate the results, letting the network choose the right scale for each pattern.

ResNet (2015)

The single most important idea in this list. ResNet added skip (residual) connections: instead of forcing a layer to learn a full transformation, it only has to learn the residual (the difference) from its input, which is then added back. This solved the vanishing-gradient problem and made 100+ layer networks trainable. Almost every modern vision backbone still uses residual connections.

EfficientNet (2019)

Asked: if I have a fixed compute budget, how should I split it between depth, width, and input resolution? EfficientNet's 'compound scaling' answers this systematically, giving strong accuracy-per-FLOP — important for anything that has to run on a robot's onboard compute.

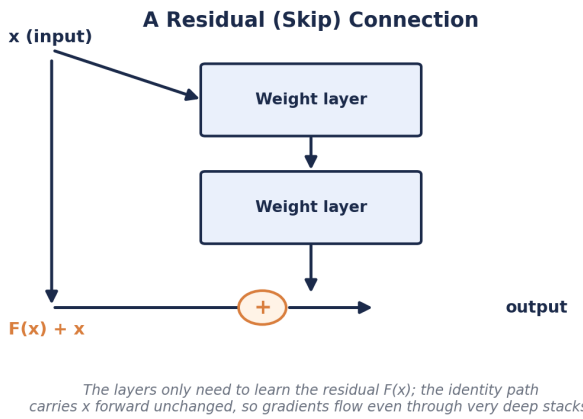


Figure 3: A residual (skip) connection: the identity path lets gradients flow through very deep stacks.

MobileNet / SqueezeNet family

Purpose-built for embedded and edge devices (exactly the kind of hardware you'd put on a mobile robot). Depthwise-separable convolutions cut compute by 8–9× with only a small accuracy loss — a very relevant family for your robotics project.

Vision Transformer — ViT (2020)

The point where the field's dominant architecture shifted from convolution to attention. Covered in full in Chapter 7.

Analogy (Detectives passing notes up a chain).

As in Chapter 1's chain of detectives: ResNet gave every detective a direct phone line to earlier detectives (skip connections), so information and corrections don't get lost or distorted after passing through twenty pairs of hands. This is why ResNet-style skip connections let you go from a handful of detectives (layers) to over a hundred without the message falling apart.

2.2 Architecture family comparison

Architecture	Year	Depth (typical)	Key innovation	Params (approx.)
LeNet-5	1998	7 layers	First working conv net	˜60K
AlexNet	2012	8 layers	ReLU, dropout, GPU training	˜60M
VGG-16	2014	16 layers	Uniform small 3×3 filters	˜138M
ResNet-50	2015	50 layers	Residual (skip) connections	˜25M
EfficientNet-B0	2019	—	Compound scaling	˜5M
MobileNetV3	2019	—	Depthwise-separable convs	˜5M
ViT-B/16	2020	12 blocks	Pure self-attention, no conv	˜86M

2.3 Transfer learning: standing on these shoulders

In practice, you rarely train one of these architectures from random weights. Instead, you take a version pretrained on a huge dataset (typically ImageNet, 1.4M images) and fine-tune it on your smaller, task-specific dataset. The early layers (edges, textures) transfer almost perfectly across tasks; only the later, more task-specific layers need significant retraining. This is why a robot vision system can often be built with only a few hundred labelled images rather than millions.

```
import torchvision.models as models
# Load a ResNet-18 pretrained on ImageNet
backbone =
models.resnet18(weights=' IMAGENET1K_V1')
# Freeze early layers, replace the final classifier
for p in backbone.parameters():
p.requires_grad = False
backbone.fc = nn.Linear(backbone.fc.in_features, num_classes)
# only backbone.fc will be trained -> fast, needs little
data
```

2.4 Case study: picking a backbone for a Jetson-class robot

Suppose you need a backbone for real-time object recognition on an NVIDIA Jetson Orin Nano-class board, at a target of 20+ FPS. Working through the checklist: ResNet-50 is accurate but too heavy for consistent 20 FPS at full resolution; VGG-16 is even heavier for its accuracy; EfficientNet-B0 or MobileNetV3 are purpose-built for this compute envelope and are the sensible starting points, with ViT-B left for a later, offline, accuracy-focused stage of the project (e.g. periodic scene understanding rather than every-frame perception).

2.5 Common pitfalls when choosing an architecture

- **Chasing benchmark leaderboard accuracy** without checking inference latency on your actual target hardware — a 2% accuracy gain is rarely worth a 5× latency cost on a mobile robot.
- **Training a large architecture from scratch** on a small dataset, when a smaller pretrained + fine-tuned model would generalise better with far less data and compute.
- **Ignoring memory footprint** — a model that technically fits in GPU memory alone may not leave enough headroom once SLAM, planning, and OS overhead are also running on the same board.

2.6 Visualising what a CNN has learned: Grad-CAM

It is often useful to see which pixels a trained CNN actually relied on for a given prediction, both for debugging (is the model looking at the object, or at a spurious background cue?) and for building trust in a deployed system. Grad-CAM (Gradient-weighted Class Activation Mapping) produces a coarse heatmap over the input image by looking at how strongly the gradients of a target class flow into the final convolutional layer's feature maps, highlighting the image regions most responsible for the prediction. On a robot, this kind of visualisation is a valuable debugging step before trusting a model's output in a safety-relevant decision, such as an obstacle-avoidance trigger.

Chapter Summary

- CNN architectures evolved by solving successive bottlenecks: depth (VGG), trainability (ResNet), and efficiency (EfficientNet/MobileNet).
- Residual connections are the single most influential idea, enabling 100+ layer networks.
- Transfer learning lets you reuse pretrained features instead of training from scratch.
- Architecture choice should always be checked against your actual latency/hardware budget, not benchmark accuracy alone.

Review Questions

1. What problem did ResNet's skip connections solve, and why did it matter for training very deep networks?
2. Why does VGGNet prefer several stacked 3×3 filters over a single larger filter?
3. What trade-off does EfficientNet's 'compound scaling' explicitly try to optimise?
4. For a robot with a constrained onboard GPU, which two architecture families would you consider first, and why?
5. What is transfer learning, and why does it typically require far less labelled data than training from scratch?
6. In the 'detectives passing notes' analogy, what specifically does a ResNet skip connection add to the chain?
7. What does a Grad-CAM heatmap show, and why is it useful before trusting a model's prediction in a safety-relevant decision?
8. Why did EfficientNet's authors scale depth, width, and resolution jointly rather than just making a network deeper?

3 Chapter 3 — Training Deep Vision Models

By the end of this chapter you will be able to:

- Choose an appropriate loss function for a given vision task.
- Explain the difference between SGD, Adam, and AdamW, and when to prefer each.
- Apply data augmentation and regularisation to reduce overfitting.
- Diagnose common training failures from their loss/accuracy curves.

An architecture is only half the story. How a network is trained — what it is told to minimise, how its weights are nudged, and what tricks keep it from memorising instead of generalising — often matters as much as the architecture itself.

3.1 Loss functions

- **Cross-entropy loss** — the standard choice for classification; penalises confident wrong predictions heavily.

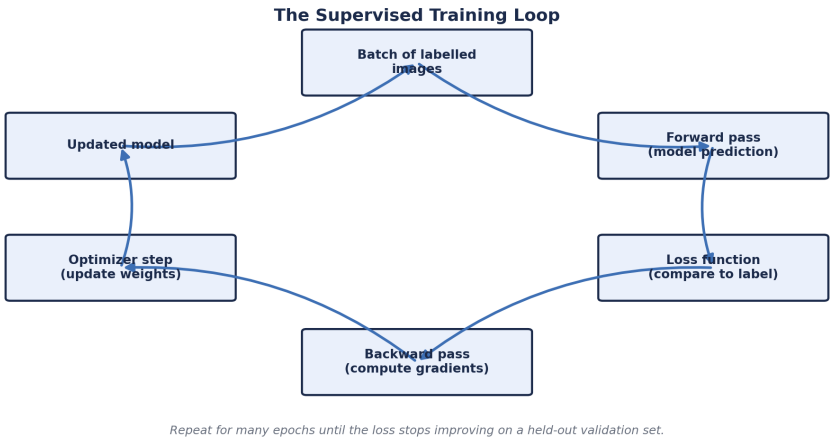


Figure 4: The supervised training loop.

- **Focal loss** — a variant used in detection, which down-weights easy examples (the abundant background) so the model focuses learning capacity on hard, rare examples (the actual objects).
- **IoU-based losses** (IoU loss, GIoU, DIoU) — directly optimise the overlap between predicted and true bounding boxes, rather than treating box coordinates as independent numbers.
- **Dice / cross-entropy for segmentation** — Dice loss handles the heavy class imbalance typical of segmentation masks (a small object against a huge background) better than plain cross-entropy alone.
- **Contrastive loss** — used by CLIP-style models (Chapter 8) to pull matching image-text pairs together and push mismatched pairs apart.

3.2 Optimizers

Optimizer	How it updates weights	When to use it
SGD (+ momentum)	Steps opposite the gradient, with inertia from past steps	Often best final accuracy for CNNs; needs careful learning-rate tuning
Adam	Adapts a per-parameter learning rate using gradient moments	Fast, robust default; very common for Transformers and quick prototyping
AdamW	Adam with decoupled weight decay	Standard choice for training ViTs and large models
RMSProp	Adapts learning rate using a moving average of squared gradients	Common in older RL and recurrent-network training

3.3 Data augmentation

Augmentation artificially expands a training set by applying label-preserving transformations, teaching the network to be invariant to things that shouldn't change the answer:

- Geometric: random crop, flip, rotation, scaling — simulates different camera viewpoints, directly relevant when a robot's camera angle varies.
- Photometric: brightness/contrast jitter, colour jitter — simulates different lighting conditions, critical for a robot moving between indoor and outdoor lighting.
- Occlusion-based: random erasing, CutMix, Mosaic (used heavily in YOLO training) — teaches robustness to partially occluded objects, common in cluttered robot workspaces.
- Domain randomisation (simulation-specific): randomising textures, lighting, and camera noise in Gazebo — covered in depth in Chapter 11, this is augmentation applied at the simulator level to close the sim-to-real gap.

3.4 Regularisation — preventing overfitting

Technique	Mechanism	Effect
Dropout	Randomly zeroes activations during training	Prevents co-adaptation of neurons, improves generalisation
Weight decay	Penalises large weights in the loss	Keeps the model simpler, less likely to memorise noise
Early stopping	Stops training when validation loss stops improving	Avoids over-fitting to the training set
Label smoothing	Softens one-hot labels slightly	Prevents the model from becoming overconfident

3.5 Learning-rate schedules

The learning rate is often the single most important hyperparameter. A common recipe: a brief warm-up phase (ramping the learning rate up from near zero) followed by a decay schedule (step decay, cosine annealing, or one-cycle) that gradually lowers it as training progresses, allowing large, fast updates early and small, precise updates near convergence.

3.6 A complete (short) training loop

```
model = TinyCNN(num_classes=10).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4,
weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt,
T_max=epochs)
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)
for epoch in range(epochs):
model.train()
for images, labels in train_loader:
images, labels = images.to(device), labels.to(device)
opt.zero_grad()
preds = model(images)
loss = loss_fn(preds, labels)
loss.backward()
opt.step()
sched.step()
val_acc = evaluate(model, val_loader) # see Chapter 6
```

```
print(f' epoch {epoch}:
val_acc={val_acc:.3f}')
```

Robotics takeaway. For a course project, train on a modest dataset captured directly from your Gazebo camera (a few hundred to a few thousand frames), lean heavily on augmentation, and start from a pretrained backbone (Section 2.3) rather than random weights — this is by far the fastest path from zero to a working perception module.

3.7 Debugging a training run

Symptom	Likely cause	What to check first
Training loss decreases, validation loss increases	Overfitting	Add augmentation, dropout, or reduce model capacity
Both losses stay high and flat	Underfitting / too-low learning rate / bug in labels	Verify labels are correct; try a higher learning rate; increase model capacity
Loss oscillates wildly	Learning rate too high, or batch size too small	Lower the learning rate; increase batch size; add gradient clipping
Validation accuracy fluctuates a lot between epochs	Validation set too small, or batch normalisation statistics unstable	Use a larger validation split; check batch size is large enough for stable BN statistics

3.8 Case study: overfitting on a small Gazebo-captured dataset

A common first failure mode: you capture 150 images of a single object from your Gazebo camera at very similar angles, train for 50 epochs, and get 99% training accuracy but the model fails immediately on real-world or differently-lit test frames. The fix follows directly from Sections 3.3–3.4: add geometric and photometric augmentation to simulate viewpoint and lighting variation the original 150 frames never captured, and start from a pretrained backbone (Chapter 2) instead of random weights so the network already 'knows' general visual structure before seeing your narrow dataset.

3.9 Mixed-precision training

Modern GPUs compute significantly faster in 16-bit floating point (FP16 or bfloat16) than in 32-bit (FP32), while most of the numerical range a network actually needs during training fits comfortably in reduced precision. Mixed-precision training keeps a master copy of weights in FP32 for stability, but performs the forward and backward passes in FP16, using automatic loss scaling to prevent small gradient values from underflowing to zero. In practice this is a near-free 1.5–3× training speedup on compatible GPUs, enabled in PyTorch with a few lines via torch.cuda.amp, and is worth turning on by default for any but the smallest training runs.

Chapter Summary

- Loss function choice should match the task: cross-entropy, focal, IoU-based, Dice, or contrastive.

- Adam/AdamW are robust defaults; SGD with momentum can reach better final accuracy with tuning.
- Augmentation and regularisation are the primary defences against overfitting.
- A cosine or warm-up learning-rate schedule is a strong default for most training runs.

Review Questions

1. What is the difference between cross-entropy loss and focal loss, and when would you prefer focal loss?
2. Explain, in one sentence each, what SGD with momentum and Adam do differently when updating weights.
3. Give two examples of geometric augmentation and two examples of photometric augmentation, and explain why a robot's camera pipeline benefits from both.
4. What does 'early stopping' protect against, and how do you decide when to stop?
5. If training loss keeps falling but validation loss starts rising, what is happening and what are two concrete fixes?
6. Why is a warm-up phase often used at the start of a learning-rate schedule?
7. What does mixed-precision training do, and why does it typically speed up training with little accuracy cost?
8. If both training and validation loss stay high and flat, what are two possible causes to check first?

This part moves beyond classification into structured outputs: where objects are (detection), their exact shape (segmentation and pose), and how to rigorously measure whether any of it actually works (evaluation).

4 Chapter 4 — Object Detection

By the end of this chapter you will be able to:

- Distinguish two-stage from one-stage object detectors and explain the accuracy/speed trade-off.
- Explain the role of anchors, IoU, and non-max suppression in a detection pipeline.
- Select an appropriate detector family for a given latency and hardware budget.
- Run inference and fine-tune a YOLO model on a custom dataset.

Classification answers 'what is in this image?' Robots almost always need more: where is it, and how many of them are there? Object detection draws a bounding box around each object instance and assigns it a label and a confidence score.

4.1 Two-stage vs one-stage detectors

- **Two-stage detectors** (R-CNN → Fast R-CNN → Faster R-CNN): first a Region Proposal Network suggests candidate boxes likely to contain an object, then a second stage classifies and refines each region. More accurate, slower.

- **One-stage detectors** (YOLO, SSD, RetinaNet): predict boxes and classes directly across the whole image in a single pass, using a dense grid of predefined anchor boxes. Faster, and the standard choice for real-time robotics.
- **Transformer / anchor-free detectors** (DETR, FCOS): treat detection as a set-prediction or per-pixel regression problem, removing hand-designed components like anchor boxes and non-max suppression.

4.2 Anchors, IoU, and non-max suppression

Anchor-based detectors tile the image with many predefined candidate boxes of different sizes and aspect ratios ('anchors'). The network predicts, for each anchor, whether it contains an object and how to adjust its coordinates.

- **Intersection over Union (IoU)** — the overlap between two boxes, computed as the area of intersection divided by the area of union. It's the standard way to decide whether a predicted box 'matches' a ground-truth box.
- **Non-max suppression (NMS)** — after prediction, many overlapping boxes may fire on the same object. NMS keeps only the highest-confidence box in each cluster of high-IoU overlapping boxes, discarding the rest.

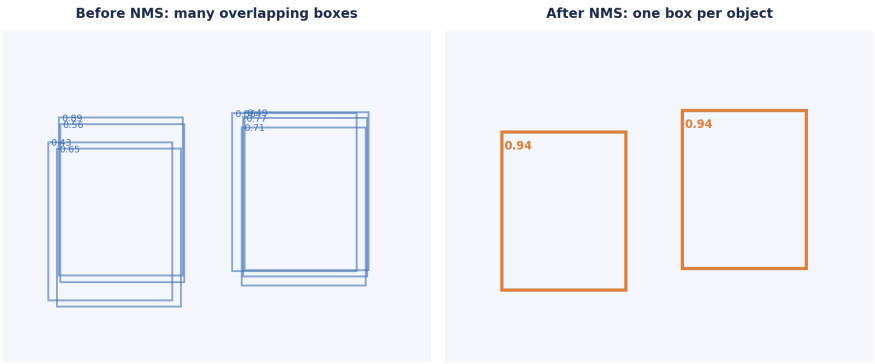


Figure 5: Non-max suppression collapses many overlapping candidate boxes into one box per object.

4.3 The YOLO family, architecturally

YOLO ('You Only Look Once') divides the image into a grid; each grid cell directly predicts a fixed number of bounding boxes, their objectness confidence, and class probabilities, all in one forward pass. Successive versions (YOLOv3 → v5 → v8 → v11) have refined the backbone (adding residual and cross-stage-partial blocks), the detection head (anchor-free in recent versions), and the training recipe (Mosaic augmentation, better loss functions), while keeping the core single-pass philosophy — which is exactly why it remains the default choice for real-time robot perception.

4.4 Speed vs accuracy: choosing a detector

Model family	Typical FPS (edge GPU)	Relative accuracy	Best fit
YOLOv8/v11	30–100+	High	Real-time robot perception, default choice
SSD / MobileNet-SSD	40–120	Medium	Very constrained onboard compute
Faster R-CNN	5–15	Very high	Offline / accuracy-critical, non-real-time
Mask R-CNN	5–10	Very high (+masks)	Precise grasp planning, bin-picking
DETR / RT-DETR	10–40	High	Cluttered scenes, fewer hand-tuned heuristics

Analogy (A spotter calling out targets).

A one-stage detector like YOLO is like a single spotter scanning a whole football field once, calling out every player's position and jersey number in one sweep. A two-stage detector is like sending an assistant to first flag 'interesting' regions, then a specialist who studies each flagged region carefully before reporting. The spotter is faster; the specialist is more careful. On a moving robot, you almost always want the spotter.

4.5 A minimal YOLO inference call (Python)

```
from ultralytics import YOLO
model = YOLO(' yolo11n.pt') # nano model:
fast, good for onboard use
results = model(' camera_frame.jpg')
for box in results[0].boxes:
cls, conf, xyxy = box.cls, box.conf, box.xyxy
print(f' class={cls} conf={conf:.2f}
bbox={xyxy}')
```

4.6 Fine-tuning a detector on your own classes

```
from ultralytics import YOLO
model = YOLO(' yolo11n.pt') # start from
pretrained COCO weights
model.train(
data=' my_robot_objects.yaml', # your
labelled dataset config
epochs=50, imgsz=640, batch=16,
)
metrics = model.val() # mAP, precision, recall - see Chapter 6
```

4.7 Common pitfalls in detection

- **Confidence threshold set too low** — floods downstream logic with false positives; too high, and real objects are missed. Tune it against a validation set, not by eye on a handful of frames.
- **Anchor sizes mismatched to object scale** — if your objects are consistently small or large relative to COCO's default anchor sizes, retrain or reconfigure anchors rather than accepting poor recall.
- **Training/deployment resolution mismatch** — a model trained at 640×640 and deployed on a resized 320×320 stream will silently lose accuracy; keep these consistent or explicitly retrain for the deployment resolution.

4.8 Case study: tuning confidence threshold for a moving robot

On a mobile robot, a detector's false positives and false negatives have asymmetric costs depending on context: an obstacle-avoidance detector should favour recall (catch every possible obstacle, tolerate some false alarms), while a pick-target detector feeding a grasp planner should favour precision (only act on confident detections, since a wrong grasp attempt wastes time or risks damage). In practice this means running the same underlying detector with two different confidence thresholds for two different downstream consumers, rather than picking one 'default' threshold for the whole system.

4.9 Worked example: computing IoU by hand

Given a predicted box with corners (x1=50, y1=50, x2=150, y2=150) and a ground-truth box with corners (x1=90, y1=90, x2=190, y2=190): the intersection region spans x from 90 to 150 and y from 90 to 150, an area of 60×60 = 3,600. Each box has area 100×100 = 10,000, so the union is 10,000 + 10,000 - 3,600 = 16,400. IoU = 3,600 / 16,400 ≈ 0.22 — below the typical 0.5 threshold, so this prediction would be counted as a miss (a false negative) despite visibly overlapping the true object, illustrating why tightening a detector's box regression, not just its classification, is often necessary to raise mAP.

Chapter Summary

- One-stage detectors (YOLO) trade some accuracy for the real-time speed robots need.
- IoU and non-max suppression are the core mechanics behind turning raw predictions into clean boxes.
- mAP@0.5:0.95 is the standard rigorous benchmark for detection quality.
- Confidence thresholds should be tuned per downstream consumer, not fixed globally.

Review Questions

1. What is the core architectural difference between a two-stage detector like Faster R-CNN and a one-stage detector like YOLO?
2. Define Intersection over Union (IoU) and explain what role it plays in non-max suppression.

3. Why might you choose SSD/MobileNet-SSD over Faster R-CNN for a robot's real-time perception, and what do you give up?
4. What problem does non-max suppression solve?
5. In the 'spotter vs specialist' analogy, which detector family does each role represent, and why?
6. Why might an obstacle-avoidance system and a grasp-target system on the same robot use different confidence thresholds for the same underlying detector?
7. Given a predicted box (20,20,120,120) and ground truth (60,60,160,160), compute the IoU and state whether it would count as a match at the standard 0.5 threshold.
8. What does the im2col-style matrix multiplication view of convolution have to do with why detectors like YOLO can run in real time on a GPU?

5 Chapter 5 — Segmentation & Pose Estimation

By the end of this chapter you will be able to:

- Distinguish semantic, instance, and panoptic segmentation, and know when each is needed.
- Explain how Mask R-CNN extends Faster R-CNN with RoIAlign and a mask branch.
- Describe how keypoint and 6-DoF pose estimation support robotic grasping.
- Walk through the pipeline from a 2D mask to an executable 3D grasp pose.

Bounding boxes are coarse — a rectangle rarely matches an object's true shape. When a robot needs to know exactly which pixels belong to an object (for precise grasping, or for building a clean map of free space), segmentation is the right tool.

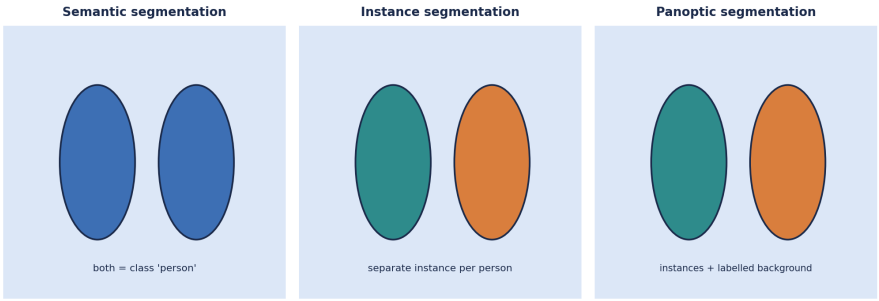


Figure 6: Semantic vs instance vs panoptic segmentation.

5.1 Three flavours of segmentation

- **Semantic segmentation** — labels every pixel with a class (e.g. 'floor', 'wall', 'person') but does not distinguish individual instances of the same class.
- **Instance segmentation** (Mask R-CNN) — gives a separate mask per object instance, so two people standing together are two masks, not one blob.

- **Panoptic segmentation** — combines both: every pixel gets a class, and 'thing' classes (objects) are separated into instances while 'stuff' classes (floor, sky, wall) remain unified regions.

5.2 How Mask R-CNN works

Mask R-CNN extends Faster R-CNN by adding a third output branch: alongside class and box prediction for each candidate region, a small fully-convolutional network predicts a binary mask for that region. The key engineering detail is RoIAlign, which extracts features for each region without the rounding-error misalignment of the original RoIPooling — that precision is exactly what makes the resulting masks pixel-accurate enough for tasks like grasp-point selection.

5.3 Foundation segmentation models

The Segment Anything Model (SAM) and successors represent a newer category: promptable segmentation. Instead of being trained on a fixed set of classes, SAM is trained to segment 'anything' given a prompt — a point, a box, or a rough mask — and generalises to object categories it never saw labelled during training. This is valuable in robotics when the set of objects a robot might encounter cannot be fully enumerated in advance.

5.4 Keypoint and pose estimation

Pose estimation predicts a fixed skeleton of keypoints (joints, corners, landmarks) rather than a full mask:

- **Human pose estimation** (OpenPose, HRNet, MediaPipe Pose) — predicts joint locations (shoulders, elbows, knees) for human-robot interaction, gesture recognition, and safety zones around a humanoid or collaborative robot.
- **Object 6-DoF pose estimation** — predicts an object's full 3D position and orientation (not just a 2D box), which is exactly what a robotic arm needs to plan a grasp: the gripper must approach at the right angle, not just the right pixel location.

5.5 From 2D mask to 3D grasp point

1. Run instance segmentation on the RGB frame to get a pixel-accurate mask of the target object.

2. Project the mask onto the aligned depth image to recover the 3D point cloud of just that object.

3. Fit a grasp-candidate estimator (geometric heuristics, or a learned model such as GG-CNN / Contact-GraspNet) to the object's point cloud.

4. Transform the resulting grasp pose from camera frame into the robot's base frame using the known camera-to-base TF transform, then send it to the motion planner.

Analogy (A tailor vs a sculptor).

A bounding-box detector is like a tailor who only measures your overall height and width. A segmentation model is like a sculptor who traces your exact outline. A pose-estimation model goes further still — like a physiotherapist marking every joint on your body, because for a robot arm

reaching to grasp something, the exact silhouette matters less than knowing precisely where to close its fingers and at what angle.

5.6 Segmentation vs pose vs detection — when to use which

Task need	Right tool
Is there an obstacle in front of me?	Object detection (fast, box is enough)
Where exactly is the free floor space?	Semantic segmentation
Pick this specific mug out of three identical ones	Instance segmentation
Grasp an object at the correct angle	6-DoF pose estimation
Track a human's arm to avoid collision	Human pose estimation

5.7 Common pitfalls in segmentation and pose

- **Treating segmentation as 'free' accuracy** — masks cost significantly more compute than boxes; only pay for pixel-level output where the downstream task genuinely needs it (e.g. grasp planning), not everywhere.
- **Ignoring mask-depth misalignment** — if the RGB and depth sensors are not properly calibrated/aligned, a pixel-accurate mask can still project onto the wrong 3D points; always verify the camera-to-depth extrinsic calibration first.
- **Assuming a fixed keypoint skeleton generalises** — a human pose model trained on standing adults may fail on seated, crouching, or partially occluded poses common in cluttered robot workspaces; validate on your actual operating conditions.

5.8 Case study: grasp point on a coffee mug

Walking through Section 5.5's pipeline concretely: instance segmentation isolates the mug's mask from the rest of the cluttered table; that mask, projected onto the aligned depth image, yields a 3D point cloud of just the mug; a grasp-candidate estimator proposes candidate gripper poses around the handle or rim (favouring the handle when available, since it's typically a more stable, collision-free grasp); the highest-scoring candidate is transformed into the robot's base frame and sent to the motion planner. Each stage can be validated independently — mask quality via Chapter 6's mIoU, grasp success rate via repeated simulated pick attempts in Gazebo before ever touching real hardware.

5.9 A note on point cloud processing

Once a masked region is projected into 3D, it becomes a point cloud — an unordered set of 3D points — rather than a structured grid like an image. Point clouds are typically cleaned with a statistical outlier filter (removing isolated noisy points) and downsampled with a voxel grid filter (reducing point density for speed) before being handed to a grasp planner. Deep architectures built directly for point clouds (PointNet, PointNet++) exist for tasks needing to reason over 3D structure beyond simple filtering, such as estimating an object's full 3D shape from a partial view.

Chapter Summary

- Segmentation gives pixel-accurate shape; pose estimation gives exact joint or object orientation.
- Instance segmentation separates individual objects; panoptic segmentation adds labelled background.
- SAM-style foundation models generalise to object categories never seen labelled during training.
- Grasp planning depends on accurate camera-to-depth calibration as much as on the model itself.

Review Questions

1. What distinguishes instance segmentation from semantic segmentation?
2. Why does Mask R-CNN need RoIAlign instead of the older RoIPooling, and what problem does it fix?
3. What is 'promptable segmentation' and how does SAM differ from a traditional fixed-class segmentation model?
4. Describe the four steps that take a 2D segmentation mask to an executable 3D grasp pose.
5. Why might a robot need both semantic segmentation (for free-space mapping) and instance segmentation (for pick-and-place) simultaneously?
6. In the 'tailor vs sculptor vs physiotherapist' analogy, which one corresponds to 6-DoF pose estimation, and why?

6 Chapter 6 — Evaluating Vision Models

By the end of this chapter you will be able to:

- Compute and interpret precision, recall, F1, and confusion-matrix entries.
- Explain how IoU and mAP quantify detection quality.
- Distinguish pixel accuracy, mIoU, and Dice for segmentation evaluation.
- Benchmark a model's latency and throughput on target hardware, not just accuracy.

A model is only as trustworthy as the metric used to validate it. This chapter covers the standard metrics for classification, detection, and segmentation, plus the latency/throughput measurements that matter specifically for real-time robot perception.

6.1 Classification metrics

- **Accuracy** — fraction of correct predictions; misleading when classes are imbalanced (e.g. 'obstacle' vs 'no obstacle' where obstacles are rare).
- **Precision** — of everything the model called positive, how much was actually positive. Matters when false alarms are costly (e.g. a robot stopping unnecessarily).
- **Recall** — of everything that was actually positive, how much did the model catch. Matters when missed detections are dangerous (e.g. failing to see an obstacle).

- **F1 score** — the harmonic mean of precision and recall, a single number balancing both.

6.2 The confusion matrix

	Predicted positive	**Predicted negative**
Actually positive	True Positive (TP)	False Negative (FN) — a miss
Actually negative	False Positive (FP) — a false alarm	True Negative (TN)

Precision = TP / (TP + FP). Recall = TP / (TP + FN). For a robot's obstacle detector, a false negative (missed obstacle) is usually far more dangerous than a false positive (unnecessary caution) — so recall is often weighted more heavily than precision in the acceptance threshold.

6.3 Detection metrics: IoU and mAP

Detection quality is judged in two steps. First, IoU (Chapter 4) decides whether a predicted box counts as a correct match for a ground-truth box, typically requiring IoU ≥ 0.5. Second, mean Average Precision (mAP) summarises precision across all recall levels and all classes into one number.

- **AP (Average Precision)** — the area under the precision-recall curve for one class.
- **mAP** — the mean of AP across all classes.
- **mAP@0.5 vs mAP@0.5:0.95** — the COCO-style metric mAP@0.5:0.95 averages mAP over IoU thresholds from 0.5 to 0.95, rewarding models with tighter, more precisely localised boxes, not just 'roughly correct' ones.

6.4 Segmentation metrics

- **Pixel accuracy** — fraction of correctly classified pixels; suffers the same imbalance problem as classification accuracy.
- **Mean IoU (mIoU)** — IoU computed between predicted and true masks, averaged across classes; the standard segmentation benchmark metric.
- **Dice coefficient** — closely related to IoU, commonly reported alongside it, especially in medical and fine-grained segmentation work.

6.5 Latency and throughput — the robotics-specific metrics

Accuracy metrics alone are not enough for a deployed robot: a highly accurate model that runs at 2 FPS is often useless for a robot moving at walking speed. Always report and budget for:

Metric	**Definition**	**Robotics rule of thumb**
Latency	Time from image capture to usable output, per frame	Should be well under your control loop period (e.g. <50 ms for a 20 Hz loop)

Metric	Definition	Robotics rule of thumb
Throughput (FPS)	Frames processed per second, sustained	Match to your camera's frame rate and the robot's maximum speed
Memory footprint	RAM/VRAM required to run the model	Must fit alongside SLAM, planning, and control processes on the same onboard computer
Power draw	Energy consumed per inference	Directly limits battery life on a mobile robot

6.6 A simple benchmarking snippet

```
import time
model.eval()
with torch.no_grad():
# warm-up
for _ in range(10):
_ = model(dummy_input)
t0 = time.time()
n = 100
for _ in range(n):
_ = model(dummy_input)
dt = (time.time() - t0) / n
print(f' latency={dt*1000:.1f} ms \textbar{}
throughput={1/dt:.1f} FPS')
```

Robotics takeaway. Always benchmark on the actual target hardware (e.g. a Jetson-class board), not just a development workstation — the gap in FPS between a desktop GPU and an embedded GPU is often 5–10×, and it is the embedded number that determines whether your perception pipeline is usable. See Chapter 13 for optimisation techniques to close that gap.

6.7 Common pitfalls in evaluation

- **Reporting accuracy on an imbalanced dataset** without checking precision/recall per class — a model that always predicts 'no obstacle' can still score high accuracy if obstacles are rare in the test set.
- **Validating on data too similar to training data** (e.g. consecutive video frames in both splits) — inflates apparent accuracy; split by scene or by time, not randomly by frame.
- **Benchmarking latency on an idle, unloaded system** — real robots run perception alongside SLAM, planning, and control; measure latency under realistic concurrent load.

6.8 Case study: computing mAP for your own dataset

Using the Ultralytics YOLO API from Chapter 4, model.val() automatically computes mAP@0.5 and mAP@0.5:0.95, along with per-class precision and recall, once you provide a validation split in the same format as training data. The practical workflow: hold out roughly 15–20% of your labelled Gazebo-captured frames as a validation set before any training begins, never used for gradient updates, and track mAP on this held-out set after every training run — this is what tells

you honestly whether a change (more data, a different backbone, more augmentation) actually helped.

6.9 Cross-validation for small datasets

When labelled data is scarce — common for a course project's own captured dataset — a single train/validation split can give a noisy, unreliable accuracy estimate that swings significantly depending on which frames happened to land in which split. k-fold cross-validation addresses this: split the data into k roughly equal parts, train k times (each time holding out a different part for validation), and average the resulting metric. This gives a more stable estimate of true performance at the cost of k× the training time — a worthwhile trade for a small dataset where a single split's evaluation noise could otherwise be mistaken for a real improvement or regression.

Chapter Summary

- Accuracy alone is misleading on imbalanced data; always check precision and recall.
- mAP and mIoU are the standard rigorous metrics for detection and segmentation respectively.
- Latency, throughput, memory, and power must be measured on real target hardware.
- A held-out validation split, never used for training, is essential for honest evaluation.

Review Questions

1. Why can accuracy alone be a misleading metric for an imbalanced dataset like obstacle detection?
2. Write out the formulas for precision and recall in terms of TP, FP, and FN, and explain in words what each measures.
3. What does mAP@0.5:0.95 reward that mAP@0.5 alone does not?
4. Why should latency be measured on the actual deployment hardware rather than a development workstation?
5. For a robot's obstacle detector, would you prioritise precision or recall, and why?
6. What three things, besides model accuracy, must be measured before a perception model is considered 'deployment ready'?

This part covers the attention-based architectures that now dominate the state of the art, from the Transformer mechanism itself through Vision Transformers to full vision-language models, closing with a practical framework for choosing between everything covered so far.

7 Chapter 7 — Attention and Transformers

By the end of this chapter you will be able to:

- Explain the mechanics of self-attention using Query, Key, and Value vectors.
- Describe how a Vision Transformer converts an image into a sequence of tokens.
- Compare the inductive biases and data requirements of CNNs versus ViTs.

- Implement a minimal multi-head self-attention block.

CNNs process an image through a fixed local window (the filter). Transformers replace that local window with a global operation called self-attention, in which every part of the input can directly relate to every other part, regardless of distance.

7.1 Self-attention, mechanically

For every input token (a word, or an image patch), the model produces three vectors: a Query (Q), a Key (K), and a Value (V), each obtained via a learned linear projection of the token's embedding. The attention weight between two tokens is the (scaled, softmax-normalised) similarity between one token's Query and another's Key; those weights are then used to combine the Values into a new, context-aware representation for every token.

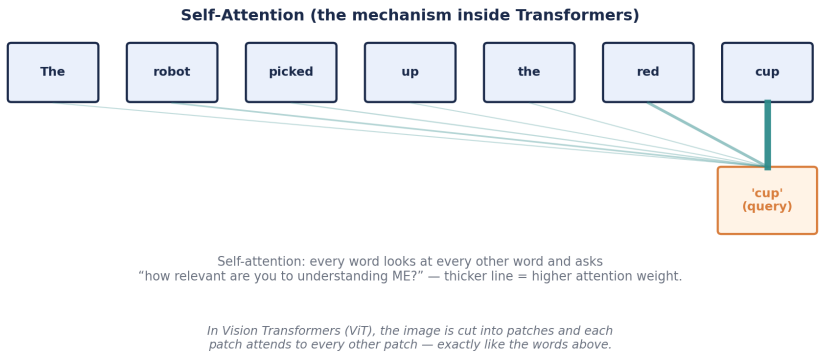


Figure 7: Self-attention: every token asks every other token 'how relevant are you to me?'

Analogy (A cocktail party).

You are at a noisy party, listening for anyone who mentions your name. Everyone is speaking at once (all tokens are 'visible' at the same time — no scanning window like a CNN), but you selectively tune in to the voices most relevant to you at that moment. Self-attention is that selective tuning, done mathematically and for every word or patch simultaneously.

7.2 Multi-head attention

Rather than computing one set of Q/K/V projections, the Transformer computes several in parallel ('heads'), each free to specialise in a different kind of relationship — one head might learn to track colour agreement, another spatial adjacency, another object-part relationships. The outputs of all heads are concatenated and linearly combined, giving the model a richer, multi-faceted notion of 'relevance' than a single attention pattern could capture.

7.3 Vision Transformer (ViT)

1. Split the image into fixed-size patches (e.g. 16×16 pixels).
2. Flatten each patch and linearly project it into an embedding vector — exactly like a word embedding.
3. Add positional embeddings, since attention has no innate sense of order or location.

4. Feed the sequence of patch embeddings through standard Transformer encoder blocks (multi-head self-attention + feed-forward).

5. A special [CLS] token's final representation is used for classification.

ViT needs far more training data than a CNN to reach the same accuracy, because it lacks the convolution's built-in assumption that nearby pixels are related (an 'inductive bias'). Given enough data, though, it scales better and now underpins most state-of-the-art vision systems.

7.4 CNN vs Transformer for vision

Property	CNN	Vision Transformer
Inductive bias	Strong (locality, translation-invariance)	Weak — must learn structure from data
Data needed	Works well on smaller datasets	Needs large-scale pretraining
Global context	Limited, grows with depth	Global from the very first layer
Compute pattern	Very GPU/edge-friendly, mature tooling	Higher memory; newer edge tooling (e.g. TensorRT ViT)
Typical robotics use	Real-time detection, embedded devices	Perception backbones for foundation/VLM-based robots

7.5 Hybrid and efficient variants

- **Swin Transformer** — restricts attention to local shifted windows, reintroducing some CNN-like locality while staying attention-based; a common backbone for detection/segmentation.
- **ConvNeXt** — a CNN redesigned using Transformer-era training tricks; matches ViT accuracy while keeping CNN efficiency.
- **MobileViT** — a lightweight hybrid explicitly designed for mobile/embedded vision — relevant if your robot's onboard compute is limited.

7.6 A minimal self-attention block

```
import torch, torch.nn as nn
class SelfAttention(nn.Module):
def __init__(self, dim, heads=8):
super().__init__()
self.heads = heads
self.qkv = nn.Linear(dim, dim * 3)
self.proj = nn.Linear(dim, dim)
def forward(self, x): # x: [batch, tokens, dim]
B, N, D = x.shape
qkv = self.qkv(x).reshape(B, N, 3, self.heads, D // self.heads)
q, k, v = qkv.permute(2, 0, 3, 1, 4) # each: [B, heads, N, d]
attn = (q @ k.transpose(-2, -1)) / (D // self.heads) ** 0.5
attn = attn.softmax(dim=-1)
out = (attn @ v).transpose(1, 2).reshape(B, N, D)
return self.proj(out)
```

7.7 Positional encoding, in a bit more depth

Because self-attention treats its input as an unordered set of tokens, the model has no innate sense that patch 3 is to the left of patch 4. Positional encodings inject this information by adding a fixed or learned vector to each token's embedding, unique to its position. The original Transformer used fixed sinusoidal functions of position; ViT and most modern vision models instead learn the positional embeddings directly during training, letting the model discover whatever positional representation best suits the task.

7.8 Common pitfalls with Transformer-based vision models

- **Training a ViT from scratch on a small dataset** — without the CNN's built-in locality assumption, a ViT trained on only a few thousand images typically underperforms a comparable CNN; always start from a pretrained checkpoint unless your dataset is very large.
- **Ignoring the quadratic cost of self-attention** — attention cost grows with the square of the number of tokens (patches), so doubling image resolution can roughly quadruple the number of patches and further increase compute; this is why Swin-style windowed attention exists.
- **Mismatched patch size at fine-tuning time** — changing patch size or input resolution after pretraining requires re-interpolating positional embeddings; most libraries handle this automatically, but it is worth verifying rather than assuming.

7.9 Relative and rotary positional encodings

The learned absolute positional embeddings described in Section 7.7 are simple but have a limitation: they don't naturally generalise to sequence lengths (or image resolutions) longer than what was seen during training. Relative positional encodings instead represent the distance between two tokens rather than each token's absolute position, and rotary positional encodings (RoPE) achieve a similar effect by rotating Query/Key vectors as a function of position, which composes cleanly with the dot-product attention computation. These variants have become common in newer large language and vision-language models because they extrapolate more gracefully to longer contexts or higher resolutions than the model was originally trained on.

Chapter Summary

- Self-attention lets every token relate directly to every other token, regardless of distance.
- ViT splits an image into patches and treats them as a token sequence for a Transformer encoder.
- ViTs need more data than CNNs but scale better and now underpin most state-of-the-art vision systems.
- Hybrid architectures (Swin, ConvNeXt, MobileViT) blend attention with CNN-like efficiency.

Review Questions

1. In self-attention, what are Query, Key, and Value vectors, and how do they combine to produce an output?
2. Why does a Vision Transformer need positional embeddings when a CNN does not need an equivalent mechanism?

3. What is multi-head attention, and why is it more expressive than a single attention computation?
4. Describe the five steps that turn a raw image into a sequence of tokens ready for a Transformer encoder.
5. Name one advantage and one disadvantage of ViT compared to a CNN of similar size.
6. In the 'cocktail party' analogy, what does a high attention weight between two words correspond to?

8 Chapter 8 — Vision-Language Models (VLMs)

By the end of this chapter you will be able to:

- Explain contrastive learning and how CLIP aligns image and text embeddings.
- Compare CLIP, BLIP-2, Flamingo, LLaVA, and GPT-4V-style models by capability.
- Use CLIP for zero-shot classification and open-vocabulary grounding.
- Apply prompt-engineering practices to improve a VLM's grounding accuracy on a robot.

A VLM connects visual understanding with natural language, so it can look at an image and answer questions about it, follow instructions referring to what it sees, or retrieve images from text — capabilities a plain classifier does not have.

8.1 CLIP — the foundation of modern VLMs

CLIP (Contrastive Language-Image Pretraining) trains an image encoder and a text encoder jointly, so that the embedding of an image and the embedding of its correct caption are pulled close together in a shared vector space, while embeddings of mismatched image-text pairs are pushed apart. This is called contrastive learning.

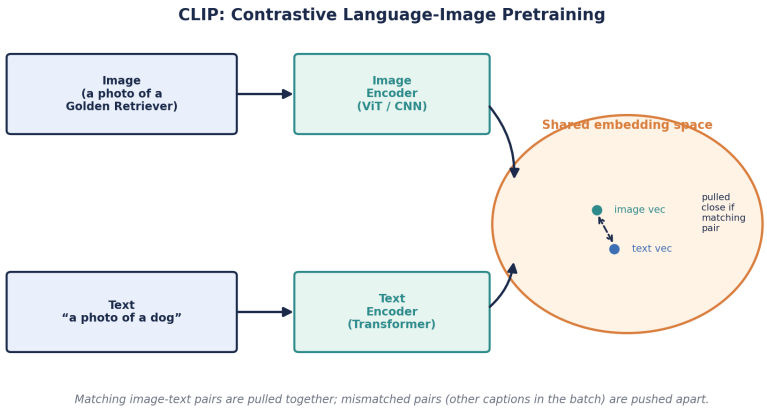


Figure 8: CLIP: Contrastive Language-Image Pretraining.

Analogy (A shared filing cabinet).

Imagine two librarians — one who only reads books (text encoder) and one who only looks at photographs (image encoder). Contrastive training forces them to file their items in the same

cabinet, in the same drawer, whenever a photo and a caption describe the same thing. Later, you can search that cabinet with either a photo or a sentence and retrieve matching items in the other form — that's exactly how CLIP enables 'zero-shot' classification: describe a class in words, and it finds matching images without ever being trained on that exact label.

8.2 From CLIP to full VLMs

Model	Core idea	Typical use
CLIP	Contrastive image-text embedding	Zero-shot classification, retrieval, as a backbone for other VLMs
BLIP / BLIP-2	Adds a captioning/generation head over CLIP-like encoders	Image captioning, visual Q&A
Flamingo	Interleaves frozen vision features into a language model via cross-attention	Few-shot visual reasoning
LLaVA	Projects vision-encoder features into an LLM's input space	Open, instruction-following visual chat
GPT-4V / Gemini / Claude (vision)	Large multimodal transformer, trained end-to-end on text+image	General-purpose visual reasoning, chart/diagram reading, robotics instruction-following

8.3 Zero-shot classification with CLIP

```
import clip, torch
model, preprocess =
clip.load(' ViT-B/32')
image = preprocess(pil_image).unsqueeze(0)
text = clip.tokenize([' a red cup',
' a robot arm', ' an empty
table'])
with torch.no_grad():
image_feat = model.encode_image(image)
text_feat = model.encode_text(text)
sims = (image_feat @ text_feat.T).softmax(dim=-1)
print(sims) # probability the image matches each phrase
```

8.4 Why VLMs matter for robotics

A VLM lets a robot be given an open-ended instruction — 'pick up the mug next to the red book' — and ground the language directly in the camera feed, without a programmer having pre-defined a fixed list of object classes. This is the basis of the current wave of 'foundation model robotics' (e.g. RT-2, OpenVLA, PaLM-E), where a single large vision-language(-action) model outputs both understanding and, increasingly, robot actions directly.

8.5 Open-vocabulary detection: bridging VLMs and Chapter 4

Open-vocabulary detectors (e.g. OWL-ViT, Grounding DINO) combine a CLIP-style text-image alignment with a detection head, so instead of a fixed class list, you provide a free-text description

at inference time ('the blue toolbox', 'a person wearing a hard hat') and get back a bounding box — the practical middle ground between Chapter 4's fast, fixed-class detectors and a full conversational VLM.

8.6 Prompt engineering for VLMs in robotics

The phrasing of a text prompt measurably affects a VLM's grounding accuracy. Practical guidelines that carry over well from language-model prompting: be specific about spatial relationships ('the cup to the left of the laptop' rather than just 'the cup'), describe distinguishing attributes when multiple similar objects are present (colour, size, material), and where the interface allows it, ask for a structured output (a bounding box, or a short list) rather than free-form prose, since structured output is far easier for a downstream planner node to parse reliably.

8.7 Case study: a VLM-guided pick instruction

Consider the instruction 'bring me the red mug from the table, not the one on the shelf.' A fixed-class detector cannot use the disambiguating clause at all; a CLIP-based open-vocabulary detector can partially use it if the phrase is passed as the query text; a full conversational VLM can reason over the whole scene and instruction jointly, correctly ruling out the shelf mug. This escalation — fixed classes → open-vocabulary phrase matching → full VLM reasoning — is a useful mental model for choosing how much 'language understanding' a given robot task actually requires, since each step up costs more latency and compute.

8.8 Common VLM grounding failure modes

- **Spatial relationship confusion** — VLMs can struggle with precise left/right or in-front/behind relationships, especially from unusual camera angles; verify grounding accuracy specifically for the spatial phrasing your robot's instructions actually use.
- **Counting errors** — general-purpose VLMs are often unreliable at exactly counting more than a handful of objects; prefer a dedicated detector's count over a VLM's count when precision matters.
- **Hallucinated objects** — a VLM may confidently describe an object that isn't actually present; cross-checking a VLM's claim against a dedicated detector's output before acting is a cheap and effective safeguard.

Chapter Summary

- CLIP aligns images and text in a shared embedding space via contrastive learning.
- Full VLMs (BLIP-2, LLaVA, GPT-4V) add captioning and instruction-following on top of CLIP-style backbones.
- Open-vocabulary detectors bridge fixed-class detection and full conversational VLM reasoning.
- VLMs let robots follow open-ended instructions without a predefined object class list.

Review Questions

1. Explain in your own words what contrastive learning does, using the 'shared filing cabinet' analogy.

2. What is zero-shot classification, and why is it possible with a model like CLIP but not with a standard supervised classifier?
3. Name three VLMs beyond CLIP and briefly state what each one adds on top of a CLIP-style backbone.
4. What is an open-vocabulary detector, and what problem does it solve that a fixed-class detector cannot?
5. Why might a robot instruction like 'the red mug, not the one on the shelf' require a full VLM rather than a simple detector?
6. Give two practical prompt-engineering tips for improving a VLM's grounding accuracy on a robot.

9 Chapter 9 — Choosing the Right Model

By the end of this chapter you will be able to:

- Apply a systematic decision checklist to any new vision task.
- Match a task's latency, hardware, and data constraints to an appropriate model family.
- Recognise when a robot needs multiple specialised models rather than one general model.
- Weigh cost-of-ownership factors alongside raw accuracy when selecting a model.

There is no single 'best' computer vision model — only the best model for a given task, latency budget, and hardware. Use the questions below as a decision checklist.

9.1 Decision checklist

1. What is the task? — classification, detection, segmentation, pose estimation, open-vocabulary/VLM reasoning.

2. What is the latency budget? — a robot's control loop often needs perception at 15–30+ Hz; a one-off scene-understanding query can take seconds.

3. What hardware will run it? — onboard embedded GPU (e.g. Jetson) vs a workstation vs cloud inference.

4. How much labelled data do you have? — little/no data favours pretrained CNNs or VLM zero-shot; lots of data can justify training or fine-tuning a Transformer.

5. Does the task require open-vocabulary understanding? — if the set of objects isn't fixed in advance, lean toward CLIP-based or VLM approaches.

9.2 A practical recommendation table

Your situation	Recommended starting point
Real-time detection on a mobile robot	YOLOv8/v11-nano or MobileNet-SSD, TensorRT-optimised

Your situation	Recommended starting point
Precise grasp/manipulation perception	Mask R-CNN or a segmentation-specific ViT (e.g. SAM) for masks + pose
Open-ended 'find the X' instructions	CLIP for zero-shot grounding, or a small open-vocabulary detector (e.g. OWL-ViT)
Learning from scratch as a course project	Start with a TinyCNN/ResNet-18 on a small dataset, then graduate to YOLO fine-tuning
Limited onboard compute (Jetson Nano-class)	MobileNet/EfficientNet-Lite backbone, INT8 quantised (see Chapter 13)
Research-grade accuracy, offline is fine	ViT/Swin backbone + DETR-style detection head

9.3 A worked example: choosing for a bipedal humanoid

Consider a humanoid robot simulation with a bipedal gait controller and a hotel-delivery task planner — close to the kind of system you may already be building. Its perception needs split naturally by task:

- **Terrain and obstacle awareness** for the gait controller: needs low latency and runs continuously → a lightweight segmentation model (MobileNet backbone) fused with depth.
- **Delivery target recognition** (finding a room number, a cart, a guest) for the task planner: runs less frequently, can tolerate more latency → a VLM or open-vocabulary detector for flexibility.
- **Human proximity / safety** around guests in a hotel corridor: needs both speed and reliability → a dedicated pose-estimation model tuned for high recall.

This illustrates the general principle: a real robot rarely uses one model for everything — it composes several models, each chosen for the latency/accuracy/vocabulary trade-off of its specific sub-task.

9.4 A note on cost of iteration

As a rule of thumb: start simple (a small CNN you train yourself, so you understand every part of the pipeline), then swap in a stronger pretrained backbone once you know exactly what your latency and accuracy requirements are. Measuring early (Chapter 6) prevents late, expensive architecture changes.

9.5 A cost-of-ownership perspective

Model choice has costs beyond raw accuracy and latency: a larger, more capable model usually also means longer training iteration cycles, more GPU memory during development, a bigger dependency footprint to maintain, and more effort to optimise for deployment (Chapter 13). For a time-boxed course project, a smaller model you fully understand and can iterate on quickly is often the better choice even when a larger model would score marginally higher on a benchmark.

9.6 Common mistakes when picking a model

- **Copying a paper's architecture** without checking whether its training data and task resemble your own — a model tuned for large-scale internet photos may transfer poorly to a

narrow robot-camera domain without fine-tuning.

- **Optimising for a leaderboard metric** that doesn't match your actual deployment metric (e.g. optimising classification accuracy when your real bottleneck is detection recall on rare classes).
- **Skipping the smallest viable baseline** — jumping straight to a large pretrained model without first trying something simple can hide whether the large model's extra complexity was ever necessary.

9.7 Ensembling: combining multiple models

When accuracy matters more than latency (e.g. an offline scene-understanding step rather than the real-time control loop), combining predictions from several different models — an ensemble — often outperforms any single model, since different architectures tend to make different, partially independent errors. A simple and effective approach is to average the class probabilities (or bounding-box coordinates, weighted by confidence) from two or three models with different backbones. The cost is roughly proportional to the number of models run, so ensembling is best reserved for non-real-time stages of a robot's perception stack, such as a periodic, careful re-verification of a grasp target before committing to the action.

Chapter Summary

- There is no universal 'best' model — only the best fit for task, latency, hardware, and data.
- Real robots typically compose several specialised models rather than relying on one.
- Starting simple and measuring early prevents costly late-stage architecture changes.
- Cost of ownership (iteration speed, maintenance) matters alongside raw benchmark accuracy.

Review Questions

1. List the five questions in the model-choice decision checklist, in your own words.
2. For a humanoid robot with both a fast gait controller and a slower task planner, why might it use two different vision models rather than one?
3. What costs, beyond accuracy and latency, should factor into a model choice for a time-boxed project?
4. Why is 'start simple, then upgrade' generally better advice than 'start with the most powerful available model'?
5. Give an example of a leaderboard metric that might not match a robot's actual deployment requirement.
6. If you have very little labelled data for a new task, which two approaches from this book would you consider first, and why?

This part brings every earlier concept into robotics: fusing vision with other sensors, connecting perception to reinforcement and imitation learning, a full ROS 2 + Gazebo project blueprint, and how to deploy a trained model within a robot's real-time and power budget.

10 Chapter 10 — Computer Vision in Robotics

By the end of this chapter you will be able to:

- Describe the full perception-to-action pipeline from sensors to actuation.
- Explain the complementary roles of cameras, depth sensors, LiDAR, and IMUs in sensor fusion.
- Distinguish feature-based, direct, and visual-inertial SLAM.
- Explain why accurate camera calibration is a prerequisite for trustworthy 3D perception.

On a robot, vision is one sensor among several, and its output must be fused with other sensors and turned into an internal model of the world before any action is taken.

10.1 The perception-to-action pipeline

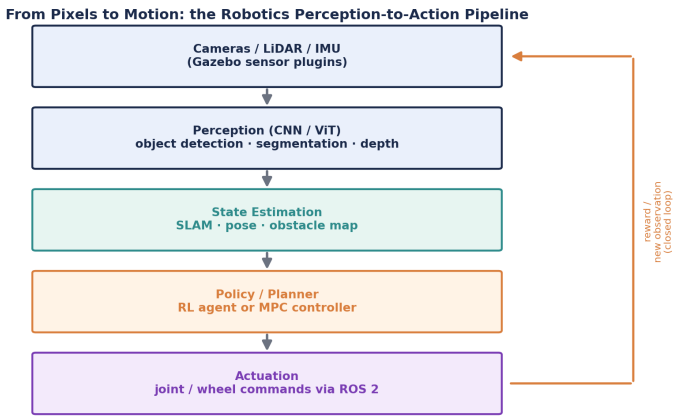


Figure 9: From pixels to motion: the robotics perception-to-action pipeline.

10.2 Sensor fusion

- **Camera (RGB)** — rich semantic information (what is it?) but no direct depth and sensitive to lighting.
- **Depth / RGB-D / stereo** — adds metric distance per pixel, essential for grasping and obstacle distance.
- **LiDAR** — precise range and geometry, works in poor lighting, but sparse and label-free (no semantics on its own).
- **IMU** — high-frequency orientation/acceleration, used to fill gaps between slower vision updates and to reject motion blur artifacts.

A typical fusion approach: run detection/segmentation on the RGB stream, project the results onto the depth or LiDAR point cloud to get 3D positions, and use the IMU to keep the pose estimate stable between vision frames (loosely- or tightly-coupled visual-inertial odometry).

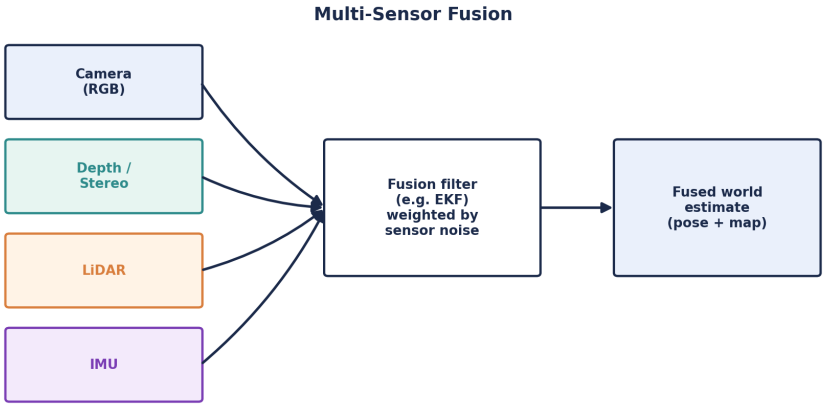


Figure 10: Multi-sensor fusion: complementary sensors combined by a fusion filter into one world estimate.

10.3 Visual SLAM

Simultaneous Localisation and Mapping (SLAM) answers two questions at once: where am I, and what does the environment around me look like? Visual SLAM (e.g. ORB-SLAM3, VINS-Fusion) uses tracked visual features across frames to estimate camera motion while incrementally building a map — the core capability behind autonomous navigation.

- **Feature-based SLAM** (ORB-SLAM3) — tracks sparse, distinctive keypoints (corners, blobs) frame to frame; efficient and mature.
- **Direct SLAM** (DSO, LSD-SLAM) — uses raw pixel intensities directly, without extracting discrete features; can be more robust in low-texture environments.
- **Visual-inertial SLAM** (VINS-Fusion) — tightly fuses camera and IMU data, giving much better robustness to fast motion and momentary visual dropout than vision alone.

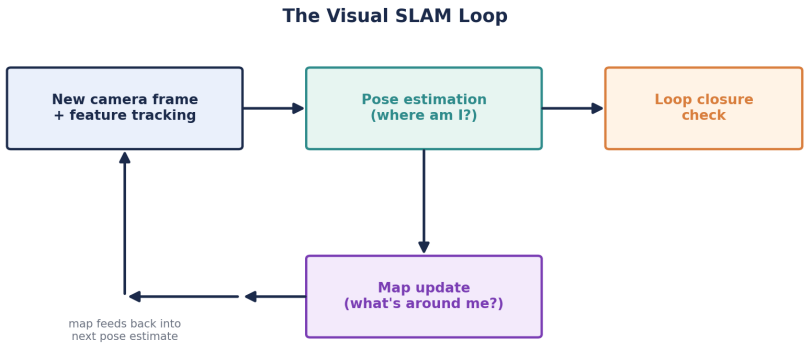


Figure 11: The visual SLAM loop: pose estimation and map update continuously feed back into each other.

10.4 Where each CV component sits in a robot stack

Robot capability	CV component used
Obstacle avoidance	Depth/segmentation + LiDAR fusion → occupancy grid
Object pick-and-place	Detection + instance segmentation → 6-DoF pose estimation
Navigation / mapping	Visual or visual-inertial SLAM
Human-robot interaction	Pose estimation, face/gesture recognition
Instruction-following ('bring me the cup')	VLM grounding + detection
Bipedal gait / terrain awareness	Depth estimation + terrain segmentation

This maps directly onto the kind of system you are already building in your **humanoid robot simulation** (ROS 2 Humble + Gazebo Harmonic, with a bipedal gait controller and a task planner) and your **mpc_nav** TurtleBot3 package — both are natural places to plug a perception module in.

10.5 The ROS 2 perception message flow

In ROS 2, a typical perception pipeline is a small graph of nodes connected by topics: a camera driver (or Gazebo sensor plugin) publishes sensor_msgs/Image, a perception node subscribes to it and publishes vision_msgs/Detection2DArray (or a custom message), and downstream planner/-controller nodes subscribe to those detections. Keeping each stage as a separate node makes the system testable in isolation — you can replay a recorded rosbag of camera frames through the perception node without needing the whole robot running.

10.6 Camera calibration: intrinsics and extrinsics

Before any 3D geometry (grasp poses, SLAM map points) can be trusted, the camera must be calibrated:

- **Intrinsic calibration** — recovers the camera's focal length, principal point, and lens distortion coefficients, usually via a checkerboard pattern captured from many angles. In Gazebo this is often provided directly by the sensor plugin's configured parameters, but should still be verified.
- **Extrinsic calibration** — recovers the fixed transform between the camera frame and the robot's base frame (or another sensor's frame, e.g. LiDAR), typically published as a static TF transform in ROS 2. Every projection from image pixels to robot-frame 3D coordinates depends on this being correct.

10.7 Common pitfalls in sensor fusion

- **Unsynchronised timestamps** — fusing a camera frame with a LiDAR scan or IMU reading captured at a meaningfully different time silently corrupts the fused estimate, especially during fast motion; use message_filters' time synchronisers in ROS 2 rather than naively pairing the 'latest' message from each topic.
- **Ignoring sensor noise characteristics** — naively averaging a precise LiDAR range with a noisy monocular depth estimate weights them equally; a proper fusion filter (e.g. an Extended Kalman Filter) should weight each sensor by its known uncertainty.

- **Stale extrinsic calibration** — if a camera is physically bumped or remounted, the extrinsic transform used in software must be re-calibrated, or every downstream 3D estimate will be silently offset.

10.8 Sharing perception across multiple robots

A fleet of robots operating in the same space (e.g. multiple delivery robots in a hotel) can share perception results rather than each independently re-detecting the same static features. A common pattern in ROS 2 is a shared map server that robots collectively update and query, plus lightweight sharing of dynamic detections (e.g. 'person detected in corridor B') over a common topic namespace. This reduces redundant computation and lets one robot benefit from what another has already observed, at the cost of needing consistent coordinate frames and clock synchronisation across the fleet.

Chapter Summary

- Perception is one stage in a longer pipeline: sensors → perception → state estimation → planning → actuation.
- Cameras, depth sensors, LiDAR, and IMUs each cover the others' blind spots when fused.
- Visual-inertial SLAM is generally the most robust choice for fast-moving robots.
- Accurate intrinsic and extrinsic calibration is a prerequisite for every downstream 3D estimate.

Review Questions

1. List the six stages of the perception-to-action pipeline shown in Figure 10.1.
2. What complementary strengths do a camera and a LiDAR bring to sensor fusion, and what does each lack on its own?
3. What is the difference between feature-based and direct visual SLAM?
4. Why does visual-inertial SLAM tend to be more robust than vision-only SLAM during fast motion?
5. What is the difference between intrinsic and extrinsic camera calibration, and why does a robot need both?
6. Why is timestamp synchronisation critical when fusing a camera frame with a LiDAR scan?

11 Chapter 11 — Connecting Vision to Reinforcement & Imitation Learning

By the end of this chapter you will be able to:

- Explain how a CNN encoder turns raw pixels into an RL-usable observation.
- Compare reinforcement learning, imitation learning, and inverse RL by data and risk trade-offs.
- Describe the sim-to-real workflow and the role of domain randomisation.
- Design a vision-derived reward signal for a robot approach or grasp task.

Perception alone doesn't move a robot. Vision must feed a decision-making system. There are three broad ways to connect the two.

11.1 Vision as the observation for an RL agent

In vision-based reinforcement learning, the raw image (or a CNN-encoded feature vector of it) is the state that a policy network receives; the policy outputs an action (e.g. joint torques or wheel velocities); the environment returns a reward. A CNN 'encoder' compresses the high-dimensional image into a compact latent vector before it reaches the policy, since learning directly from raw pixels is sample-inefficient.

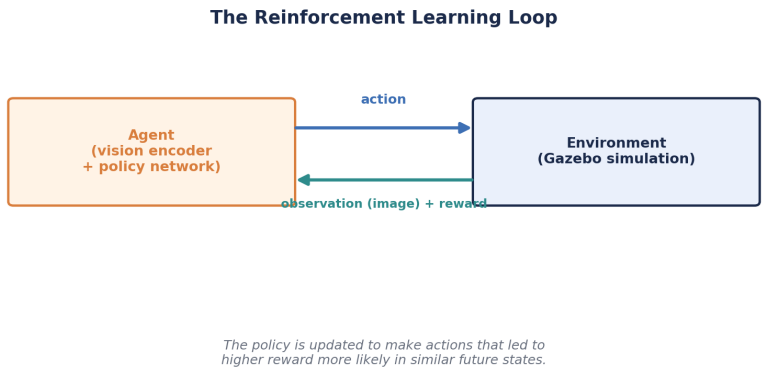


Figure 12: The reinforcement learning loop.

Analogy (Learning to ride a bike by feel, not by rulebook).

In classical robotics you might hand-code 'if obstacle within 0.5 m, turn'. In vision-based RL, the robot is more like a child learning to ride a bike: it tries actions, feels which ones lead to good outcomes (staying upright, reaching the goal) versus bad ones (crashing), and gradually tunes its own internal 'rules' — without anyone ever writing those rules down explicitly.

11.2 Reinforcement learning vs. imitation learning

Approach	How it learns	Pros	Cons
Reinforcement Learning (RL)	Trial-and-error, maximises a reward signal	Can discover novel strategies; no expert data needed	Sample-inefficient; reward design is hard; risky on real hardware
Imitation Learning / Behaviour Cloning	Supervised learning on expert demonstrations (state → action)	Fast to train; safe, no exploration on hardware	Limited to what the expert demonstrated; compounding errors

Approach	How it learns	Pros	Cons
Inverse RL	Infers the reward function from expert demonstrations, then runs RL	Generalises better than pure cloning	Computationally expensive; harder to implement
Sim-to-real RL	Trains in simulation (e.g. Gazebo/Isaac Sim), transfers to hardware	Unlimited safe trial-and-error, parallelisable	'Reality gap' — sim physics/visuals differ from the real world

11.3 Common RL algorithms used with visual observations

Algorithm	Family	Notes for vision-based robotics
PPO (Proximal Policy Optimisation)	On-policy, policy-gradient	Stable, widely used default for continuous-control robot tasks
SAC (Soft Actor-Critic)	Off-policy, actor-critic	Sample-efficient, good for continuous action spaces like joint torques
DQN and variants	Off-policy, value-based	Suited to discrete action spaces (e.g. turn left/right/forward)
DreamerV3 / model-based RL	Model-based	Learns a world model from images, very sample-efficient but complex to implement

11.4 The sim-to-real workflow (what you'll actually do)

1. Build the environment and robot model in Gazebo (URDF/Xacro, camera + depth sensor plugins).

2. Train a CNN-based perception module (or use a pretrained YOLO/segmentation model) on simulated camera frames.

3. Feed the perception output (or the encoded image) as part of the RL observation, alongside proprioceptive state (joint angles, IMU).

4. Train the policy in simulation with a framework such as Stable-Baselines3, RLlib, or Isaac Lab, using domain randomisation (randomising textures, lighting, camera noise) to reduce the reality gap.

5. Evaluate in simulation, then deploy to hardware (or a more realistic simulator) and fine-tune if needed.

11.5 Reward shaping from vision

Vision-derived quantities make excellent reward terms: distance-to-goal from a detected object's bounding box centre, a penalty when a segmentation mask shows the gripper overlapping an obstacle, or a bonus when a detector confirms the target object is grasped. This is exactly how you would turn your existing perception pipeline into training signal for an RL policy.

11.6 A minimal Gymnasium-style wrapper around a Gazebo camera observation

```
import gymnasium as gym
import numpy as np
class GazeboVisionEnv(gym.Env):
def __init__(self):
self.observation_space = gym.spaces.Box(
low=0, high=255, shape=(84, 84, 3), dtype=np.uint8)
self.action_space = gym.spaces.Box(
low=-1.0, high=1.0, shape=(2,), dtype=np.float32) # e.g. linear/angular
vel
def step(self, action):
self._publish_cmd_vel(action) # ROS 2 publisher
obs = self._get_camera_frame() # from /camera/image_raw
reward = self._compute_reward(obs) # e.g. distance to detected goal
terminated = self._check_done()
return obs, reward, terminated, False, {}
```

Robotics takeaway. Your existing mpc_nav package (MPC-based trajectory tracking with LiDAR obstacle avoidance) is a strong classical baseline. A natural extension is to keep MPC for low-level trajectory tracking while an RL or VLM-based layer decides higher-level goals from vision — a hybrid architecture that is often more practical than replacing the controller entirely.

11.7 Common pitfalls in vision-based RL

- **Reward hacking** — an agent finds an unintended way to maximise reward that technically satisfies the reward function but not the actual goal (e.g. spinning in place to keep a target barely in frame rather than approaching it); watch recorded rollouts, not just the reward curve.
- **Sparse rewards** — a reward given only on task completion (e.g. 'grasped the object') gives almost no learning signal early in training; shape intermediate rewards (e.g. decreasing distance-to-target) to guide early exploration.
- **Overfitting to the simulator's visuals** — a policy trained on a narrow range of simulated textures/lighting may fail to generalise even to a slightly different simulated scene, let alone the real world; domain randomisation (Section 11.4) directly addresses this.

11.8 Case study: reward shaping for an approach task

For a 'approach and face the detected object' task, a sparse reward ('+1 if within 0.3 m and facing the object, 0 otherwise') is slow to learn from. A shaped reward instead combines several vision-derived terms every step: a small positive term proportional to the decrease in distance to the object's estimated 3D position (from Chapter 5's projection pipeline), a small penalty proportional to angular misalignment with the object, and a larger terminal bonus on successful approach. This gives the policy continuous feedback at every timestep rather than only at the end of an episode, dramatically speeding up learning.

11.9 Curriculum learning

Rather than training a policy directly on the hardest version of a task, curriculum learning starts with an easier variant (e.g. the target object always in clear view, close range) and gradually increases difficulty (occlusions, longer distances, cluttered backgrounds) as the policy's success rate improves. This mirrors how a human learner would practise, and often converges faster and more reliably than training on the full task difficulty from the start, particularly for vision-based tasks where early random exploration is unlikely to ever stumble onto a successful outcome in a hard scene.

Chapter Summary

- Vision-based RL uses a CNN/ViT encoder to compress pixels into a usable policy observation.
- Imitation learning is safer and faster to train; RL can discover novel strategies but needs far more data.
- Sim-to-real transfer relies heavily on domain randomisation to close the reality gap.
- Vision-derived quantities (distance, mask overlap, detection confidence) make effective reward terms.

Review Questions

1. In vision-based RL, what role does a CNN encoder play before the policy network sees the observation?
2. What is the key difference between reinforcement learning and imitation learning in terms of what data each requires?
3. Why is sim-to-real transfer difficult, and what technique is commonly used to reduce the 'reality gap'?
4. Describe two ways a vision-derived signal (e.g. from a detector or segmentation mask) could be turned into a reward term.
5. What is 'reward hacking', and how would you notice it was happening during training?
6. Why might a robotics team choose a hybrid architecture (classical MPC plus a learned high-level layer) over a fully learned end-to-end policy?

12 Chapter 12 — Mini-Project Blueprint: CV-Enabled Robot in ROS 2 + Gazebo

By the end of this chapter you will be able to:

- Lay out a complete ROS 2 + Gazebo perception project architecture.
- Implement a working perception node that publishes detections and an annotated image stream.
- Follow a milestone-based build order from a live camera feed to a closed perception-action loop.
- Debug a ROS 2 perception pipeline using rosbag replay, TF tools, and topic diagnostics.

This chapter is a concrete, from-scratch build plan for a single-robot computer-vision project using ROS 2 Humble and Gazebo Harmonic — the same stack as your existing humanoid simulation and mpc_nav packages — so you can reuse infrastructure you already have.

12.1 Project architecture

- Gazebo Harmonic — physics simulation + a simulated RGB-D camera sensor plugin publishing to ROS 2 topics.
- ROS 2 Humble — the middleware connecting perception, planning, and control nodes via topics/services/actions.
- A perception node — subscribes to /camera/image_raw, runs a CNN/YOLO model, publishes detections + a processed image.
- RViz2 — visualises the camera feed, detections, TF frames, and the point cloud/occupancy map.
- A planner/controller node — consumes detections (e.g. via your existing mpc_nav trajectory tracker) to act on what is seen.

12.2 Step-by-step build order

1. Set up the workspace: colcon workspace with a dedicated package, e.g. cv_bot_perception.

2. Add a camera to your robot's URDF/Xacro using the gazebo_ros_camera (or depth camera) plugin, publishing sensor_msgs/Image and CameraInfo.

3. Verify raw video: ros2 run rqt_image_view rqt_image_view, subscribed to the camera topic, to confirm Gazebo is publishing frames correctly.

4. Write the perception node in Python: subscribe to the image topic via cv_bridge, run inference (start with a pretrained YOLO model, or your own TinyCNN if the goal is truly 'from scratch'), and publish results as a custom or vision_msgs/Detection2DArray message.

5. Visualise in RViz2: add an Image display for the annotated frame and a Marker/BoundingBox display for detections.

6. Close the loop: feed detections into your controller (e.g. 'turn toward the detected object', or extend mpc_nav to treat a detected obstacle as a dynamic constraint).

7. (Optional, RL extension) Wrap the Gazebo environment with a Gymnasium-style interface and train a policy that uses the perception node's output as part of its observation, following Chapter 11.

12.3 Minimal ROS 2 perception node (Python)

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
from ultralytics import YOLO
class PerceptionNode(Node):
```

```
def __init__(self):
super().__init__(' cv_bot_perception')
self.bridge = CvBridge()
self.model = YOLO(' yolo11n.pt')
self.sub = self.create_subscription(
Image, '/camera/image_raw',
self.on_frame, 10)
self.pub = self.create_publisher(Image,
'/perception/annotated', 10)
def on_frame(self, msg: Image):
frame = self.bridge.imgmsg_to_cv2(msg,
' bgr8')
results = self.model(frame, verbose=False)
annotated = results[0].plot()
out = self.bridge.cv2_to_imgmsg(annotated,
' bgr8')
self.pub.publish(out)
def main():
rclpy.init()
rclpy.spin(PerceptionNode())
if __name__ == '__main__':
main()
```

12.4 Package layout

```
cv_bot_perception/
|-- -- package.xml
|-- -- setup.py
|-- -- cv_bot_perception/
| |-- -- __init__.py
| `-- -- perception_node.py
|-- -- launch/
| `-- -- perception.launch.py
`-- -- config/
`-- -- camera_params.yaml
```

12.5 From-scratch option

If the goal is genuinely to build the CNN from scratch (rather than using a pretrained YOLO), replace the model line with your own TinyCNN/ResNet-style network trained on a small labelled dataset captured from the Gazebo camera itself — a good way to validate Chapter 1's concepts end-to-end before relying on a pretrained detector for the rest of the pipeline. The full training loop from Chapter 3, plus the evaluation code from Chapter 6, is directly reusable here.

12.6 Suggested milestones

Milestone	What 'done' looks like
1. Camera live in Gazebo	rqt_image_view shows a real-time feed from the simulated camera
2. Perception node running	Annotated detections published and visible in RViz2

Milestone	What 'done' looks like
3. Closed-loop reaction	Robot visibly turns/stops/approaches based on a detection
4. Benchmarked	Latency and mAP measured and recorded (Chapter 6)
5. (Stretch) RL policy	A trained policy that uses perception output beats a random baseline in simulation

12.7 Testing and debugging the pipeline

- **Test nodes in isolation** — record a rosbag of just the camera topic during a Gazebo run, then replay it (ros2 bag play) against the perception node alone, without the rest of the simulation running, to iterate faster on the model.
- **Check TF frames first** when 3D positions look wrong — ros2 run tf2_tools view_frames generates a diagram of the current transform tree, which is often the fastest way to spot a missing or misconfigured frame.
- **Use ros2 topic hz** on both the raw camera topic and the detections topic to confirm the perception node isn't silently falling behind the camera's publish rate.

12.8 Extending the project further

1. Add a second camera or depth sensor and fuse detections across viewpoints for more robust coverage (Chapter 10).

2. Replace the pretrained YOLO model with your own trained-from-scratch network for the specific objects in your simulation, following the workflow of Chapter 3.

3. Wrap the perception + controller loop as a Gymnasium environment and train an RL policy on top of it, following Chapter 11.

4. Export the trained model to TensorRT (Chapter 13) and benchmark the full pipeline's latency, not just the model's.

12.9 Continuous integration for a ROS 2 perception package

As the project grows, a lightweight CI setup pays for itself quickly: a GitHub Actions (or similar) workflow that builds the colcon workspace and runs a small suite of unit tests on every push catches broken imports, message-type mismatches, and launch-file errors before they reach a live Gazebo session. A practical minimum test suite includes a launch test that starts the perception node and asserts it publishes to the expected topic within a timeout, plus a few pure-Python unit tests for any non-ROS helper functions (e.g. the mask-to-grasp-pose transform math), which can run without ROS or Gazebo at all and therefore execute in seconds.

Chapter Summary

- A minimal perception project needs just five pieces: simulator, middleware, perception node, visualiser, and controller.
- Building milestone-by-milestone (camera → perception → closed loop → benchmark) de-risks the project.

- Isolated node testing via rosbag replay speeds up iteration dramatically.
- The from-scratch option lets you validate Chapter 1–3's concepts before relying on pretrained models.

Review Questions

1. List the five core components of the project architecture described in Section 12.1.
2. What ROS 2 message type is used to publish a set of detections, and what does it typically contain per detection?
3. Why is it useful to record a rosbag of camera frames rather than always testing against a live Gazebo simulation?
4. What does 'ros2 run tf2_tools view_frames' help you debug?
5. What is the purpose of the 'from-scratch option' described in Section 12.5, given that a pretrained YOLO model is available?
6. Name two ways this mini-project could be extended toward the reinforcement-learning material in Chapter 11.

13 Chapter 13 — Deployment & Edge Optimisation

By the end of this chapter you will be able to:

- Explain quantisation, pruning, and knowledge distillation as model-shrinking techniques.
- Export a model through ONNX to a TensorRT engine for embedded deployment.
- Benchmark and compare FP32, FP16, and INT8 deployment options.
- Apply a deployment checklist before shipping a perception model to a robot.

A model that scores well in a notebook is not automatically usable on a robot. This chapter covers the standard techniques for shrinking and accelerating a model so it runs within a robot's real-time and power budgets.

13.1 Quantisation

Quantisation reduces the numerical precision of a model's weights and activations — typically from 32-bit floating point (FP32) down to 16-bit float (FP16) or 8-bit integer (INT8) — shrinking model size and increasing inference speed, usually with a small, controllable accuracy cost.

- **Post-training quantisation** — quantise an already-trained model directly; fast to apply, small accuracy loss for most CNNs.
- **Quantisation-aware training** — simulates quantisation effects during training, so the model learns to be robust to the lower precision; better accuracy retention, more setup effort.

13.2 Pruning and distillation

- **Pruning** — removes weights or entire channels that contribute little to the output, directly shrinking the model.

- **Knowledge distillation** — trains a small 'student' network to mimic the outputs of a large, accurate 'teacher' network, often recovering more accuracy per parameter than training the small network from scratch.

13.3 ONNX and TensorRT

ONNX (Open Neural Network Exchange) is a common intermediate format: export a trained PyTorch model to ONNX, and it becomes runnable by many optimised inference engines without needing PyTorch itself installed on the robot. NVIDIA TensorRT then takes an ONNX model and compiles it into a highly optimised engine specific to the target GPU (e.g. a Jetson board), fusing layers and selecting the fastest available kernels and precision automatically.

```
\begin{figure}[h]
\centering
\includegraphics[width=0.72\textwidth]{img/196bf2b95549c1ecaed348679110c317338d377b.png}
\caption{The model export and quantisation pipeline, from a
trained PyTorch model to an optimised on-robot engine.}
\end{figure}
# 1. Export PyTorch -> ONNX
torch.onnx.export(model, dummy_input,
' model.onnx',
input_names=[' input'],
output_names=[' output'],
opset_version=17)
# 2. Build a TensorRT engine (on the Jetson / target device)
# trtexec --onnx=model.onnx --saveEngine=model.trt --fp16
# 3. Run inference with the TensorRT engine at deploy time
# (via the tensorrt Python API or a ROS 2 TensorRT inference node)
```

13.4 Benchmarking on Jetson-class hardware

Format / precision	Relative size	Relative latency	Typical accuracy drop
PyTorch FP32 (baseline)	1.0×	1.0×	—
ONNX Runtime FP32	1.0×	˜0.7–0.9×	None
TensorRT FP16	˜0.5×	˜0.3–0.5×	Negligible
TensorRT INT8	˜0.25×	˜0.15–0.3×	Small, must validate per-task

13.5 A practical deployment checklist

1. Profile the full pipeline (not just the model) — image capture, pre-processing, and post-processing (NMS, mask decoding) often dominate latency more than the network forward pass itself.

2. Quantise to FP16 first (almost always safe); move to INT8 only if you still need the extra speed, and validate accuracy afterward using Chapter 6's metrics.

3. Batch size 1 for real-time robot inference — optimise for single-frame latency, not throughput on large batches.

4. Keep a fallback: a simpler, cheaper model (or a classical CV heuristic) that can run if the primary model's latency spikes unexpectedly.

Robotics takeaway. For an onboard Jetson-class device, a YOLO-nano model exported to TensorRT FP16 is a strong, well-trodden default that comfortably clears real-time requirements for a mobile robot's obstacle and object detection needs.

13.6 Common pitfalls in deployment

- **Validating accuracy only in the training framework** — always re-measure accuracy (Chapter 6) after exporting and quantising, since ONNX/TensorRT conversion can occasionally change numerical behaviour subtly.
- **Forgetting pre/post-processing in the exported graph** — resizing, normalisation, and NMS are sometimes left outside the exported model; make sure the deployed pipeline reproduces them exactly, or accuracy will silently degrade.
- **Assuming INT8 is always safe** — some layers (e.g. in segmentation heads) are more sensitive to precision loss than others; validate per-task rather than assuming a blanket INT8 conversion is harmless.

13.7 Case study: shrinking a segmentation model for Jetson Nano

A Mask R-CNN model that runs comfortably on a desktop GPU is typically far too slow for a Jetson Nano-class board. A practical shrinking path: swap the backbone for a MobileNet variant (Chapter 2), export to ONNX, convert to TensorRT at FP16, and if still too slow, move the segmentation head itself to a lighter architecture (e.g. a compact U-Net) rather than pushing INT8 quantisation on a model whose masks are already precision-sensitive. Each step should be validated with the mIoU metric from Chapter 6 to confirm the accuracy loss is acceptable for the task.

13.8 Model versioning and rollback

Once a perception model is deployed on a robot, treat it with the same version discipline as code: tag every exported model artefact with the training data version, architecture, and evaluation metrics it was built from, and keep the previous known-good model available for immediate rollback. A model that regresses silently in a new corner case (a lighting condition, an object variant) is far easier to recover from if you can revert to yesterday's checkpoint in seconds rather than needing to retrain under time pressure.

Chapter Summary

- Quantisation (FP16/INT8) is usually the highest-value, lowest-effort deployment optimisation.
- ONNX plus TensorRT is the standard path from a trained PyTorch model to an embedded-optimised engine.
- Always re-validate accuracy after export and quantisation, never assume it transfers unchanged.
- Optimise for single-frame latency, not large-batch throughput, on a real-time robot.

Review Questions

1. What is the difference between post-training quantisation and quantisation-aware training?
2. Explain, in one sentence, what knowledge distillation does.
3. What is ONNX, and why is it useful as an intermediate format before deploying to TensorRT?
4. From the benchmarking table in Section 13.4, roughly how much latency reduction does TensorRT FP16 typically provide over a PyTorch FP32 baseline?
5. Why should you re-measure accuracy after quantisation rather than trusting the pre-quantisation number?
6. Why is batch size 1 usually the right optimisation target for a robot's real-time perception, rather than large-batch throughput?

14 Chapter 14 — Platforms & Tooling Comparison

By the end of this chapter you will be able to:

- Compare Gazebo, Isaac Sim, Webots, MuJoCo, and Unity ML-Agents for robotics simulation.
- Choose between Python, C++, and Rust for different ROS 2 node roles.
- Select appropriate visualisation and dataset-labelling tools for a robotics CV project.
- Assemble a coherent toolchain recommendation for your own robot project.

14.1 Simulators

Simulator	Physics/rendering	ROS 2 integration	Best for
Gazebo (Harmonic)	Good physics, open-source	Native, mature	General robotics, your current stack
NVIDIA Isaac Sim	Photorealistic (RTX), GPU-accelerated	Via ROS 2 bridge	Large-scale RL, sim-to-real, synthetic data
Webots	Good physics, easier UI	Supported	Education, quick prototyping
MuJoCo	Very accurate contact physics	Via wrappers	RL research, locomotion/manipulation
Unity ML-Agents	Game-engine rendering	Via bridges	Visually rich RL environments

14.2 Middleware & languages

Your stack — ROS 2 Humble — is the right default: it is the industry-standard middleware for exactly this kind of multi-node perception/planning/control system, with mature Gazebo and RViz2 integration.

- **Python** (rclpy) — fastest to iterate, huge ML ecosystem (PyTorch, Ultralytics); use this for perception nodes.

- **C++** (rclcpp) — use for tight control loops needing low, deterministic latency.
- **Rust** (via crates such as r2r or ros2-rust) — an emerging option in the ROS 2 ecosystem, offering C++-level performance with memory safety; still less mature tooling/model support than Python for the CV/ML side, so it's best used for safety-critical control nodes rather than the perception/model-inference stage.

14.3 Visualisation

- **RViz2** — the standard ROS 2 tool for viewing camera feeds, TF frames, point clouds, and detections in real time.
- **rqt_image_view** — lightweight, quick way to sanity-check a raw or annotated camera topic.
- **Foxglove Studio** — a modern, browser-based alternative to RViz2 with richer plotting and log/bag replay.

14.4 Dataset and labelling tools

- **Roboflow / CVAT / Label Studio** — for hand-labelling boxes, masks, and keypoints on frames captured from your Gazebo camera or a real one.
- **Synthetic data from Gazebo/Isaac Sim** — since ground-truth labels (exact object poses, masks) are known inside the simulator, you can generate large labelled datasets automatically, which is often faster and cheaper than manual annotation for a course project.

14.5 Suggested toolchain for your project

Given your existing ROS 2 Humble + Gazebo Harmonic humanoid simulation and mpc_nav package, the lowest-friction path is: Gazebo Harmonic for simulation → Python/rclpy perception node (Ultralytics YOLO or your own TinyCNN) → RViz2 for visualisation → your existing MPC/planner for downstream control, with an optional Gymnasium-wrapped Gazebo environment if you extend into reinforcement learning, and TensorRT export (Chapter 13) once you move toward real hardware.

14.6 Migration and version considerations

Robotics tooling moves quickly, and version mismatches are a common source of wasted debugging time. When following this book: confirm you are using ROS 2 Humble (an LTS release) rather than a newer non-LTS distribution unless you have a specific reason to; confirm your Gazebo version is Harmonic (the version paired with Humble in most current tutorials) rather than the older, differently-structured Gazebo Classic, since plugin APIs and SDF syntax differ meaningfully between them; and pin your Python ML library versions (PyTorch, Ultralytics) in a requirements file, since minor version changes occasionally alter default behaviour.

14.7 Final checklist before you start building

- ROS 2 Humble installed and sourced; a working colcon workspace.
- Gazebo Harmonic installed, with your robot's URDF/Xacro spawning correctly and a camera plugin publishing images.

- Python environment with PyTorch, Ultralytics, and cv_bridge available inside the ROS 2 workspace.
- RViz2 configured with an Image display and ready to subscribe to your perception node's output.
- A clear decision, from Chapter 9, on which model family you are starting with and why.

14.8 Cloud vs edge inference

Not every perception workload needs to run onboard. For latency-critical control-loop perception (obstacle detection, grasp targeting), onboard edge inference (Chapter 13) is essential. For heavier, less time-critical reasoning — a periodic VLM scene-understanding query, or an offline model retraining job — offloading to a cloud or local workstation GPU over a network connection can be perfectly acceptable, trading network latency and reliability for far greater compute. A well-designed robot perception stack often mixes both: fast, small models onboard for the control loop, and an optional cloud/workstation call for occasional, richer understanding when connectivity allows.

Chapter Summary

- Gazebo Harmonic + ROS 2 Humble is the right default stack for this book's projects.
- Isaac Sim, Webots, and MuJoCo each trade off differently on realism, ease of use, and RL throughput.
- Python is fastest for perception iteration; C++ suits tight control loops; Rust suits safety-critical nodes.
- Version consistency (Humble + Harmonic, pinned Python libraries) avoids a large class of avoidable bugs.

Review Questions

1. Why is Gazebo Harmonic (rather than Gazebo Classic) the right choice to pair with ROS 2 Humble for a new project?
2. What is the main advantage of Isaac Sim over Gazebo for large-scale reinforcement learning, and what does Gazebo offer in return?
3. When would you choose C++ (rclcpp) over Python (rclpy) for a ROS 2 node in this project?
4. What role does Foxglove Studio play that complements RViz2?
5. Why is it useful to generate synthetic labelled data directly from a simulator rather than always hand-labelling real images?
6. Summarise, in your own words, the suggested toolchain for the mini-project in one or two sentences.

Quick-Reference Summary

Level	What you learn	Key models
Foundations	Convolution, pooling, hierarchical features, training mechanics	TinyCNN, LeNet
Classical deep CV	Deeper, more efficient CNNs	AlexNet, VGG, ResNet, EfficientNet
Structured prediction	Where and what shape is the object, and how to evaluate it	YOLO, Faster R-CNN, Mask R-CNN, DETR
Attention-based	Global context via self-attention	ViT, Swin, ConvNeXt
Multimodal / VLM	Vision grounded in language	CLIP, BLIP-2, LLaVA, GPT-4V/Gemini
Applied to robots	Perception → decision → action, deployed on real hardware	Sensor fusion, visual SLAM, RL/imitation learning, TensorRT

Recommended learning path: build the TinyCNN by hand (Ch.1) → fine-tune a YOLO model (Ch.4) → read and reproduce a ViT forward pass (Ch.7) → experiment with CLIP zero-shot (Ch.8) → wire a perception node into your existing Gazebo + ROS 2 + mpc_nav stack (Ch.12) → extend to a vision-based RL policy (Ch.11) → optimise for deployment (Ch.13).

Glossary of Terms

Alphabetical short definitions of terms used throughout this book, for quick lookup.

Term	Definition
6-DoF pose	An object's full position and orientation in 3D space (3 translation + 3 rotation degrees of freedom).
Anchor box	A predefined candidate bounding box of a given size/aspect ratio, tiled across the image, used as a reference for anchor-based detectors.
Attention (self-attention)	A mechanism where each token computes a weighted combination of all other tokens, weighted by learned relevance.
Backbone	The feature-extraction part of a vision model (often a pretrained CNN or ViT) that downstream task-specific heads build on.
Batch normalisation	A layer that normalises activations within a training batch, stabilising and speeding up training.
Batch size	The number of samples processed together in one forward/backward pass during training.
CLIP	Contrastive Language-Image Pretraining; a model that aligns image and text embeddings in a shared space.
Confusion matrix	A table breaking predictions into true/false positives/negatives, the basis for precision and recall.
Contrastive learning	A training method that pulls matching pairs together and pushes mismatched pairs apart in embedding space.
Convolution	An operation sliding a small learnable filter over an input to produce a feature map.

Term	Definition
Data augmentation	Applying label-preserving transformations to training data to improve generalisation.
Domain randomisation	Randomising simulation parameters (textures, lighting, noise) during training to improve transfer to the real world.
Dropout	A regularisation technique that randomly zeroes activations during training to prevent co-adaptation.
Epoch	One complete pass through the entire training dataset.
Extrinsic calibration	Recovering the fixed geometric transform between a sensor's frame and another reference frame.
Feature map	The output of applying a convolution filter across an input, forming a 2D grid of response values.
Fine-tuning	Continuing to train a pretrained model on a new, typically smaller, task-specific dataset.
Focal loss	A loss variant that down-weights easy examples so training focuses on hard, rare cases.
Imitation learning	Learning a policy by supervised training on expert demonstrations (state to action pairs).
Inductive bias	Built-in assumptions an architecture makes about the data (e.g. CNNs assume locality).
Intrinsic calibration	Recovering a camera's internal optical parameters: focal length, principal point, distortion.
IoU (Intersection over Union)	The overlap between two regions, divided by their union; used to judge detection/segmentation correctness.
Knowledge distillation	Training a small model to mimic a larger model's outputs, recovering more accuracy per parameter.
Latency	The time taken from input capture to a usable model output for a single sample.
Learning rate	The step size used when updating model weights during training.
Loss function	The quantity a model is trained to minimise, measuring prediction error.
mAP (mean Average Precision)	A detection-quality metric averaging precision across recall levels and classes.
Mask R-CNN	A detection architecture extended with a per-region mask-prediction branch for instance segmentation.
Multi-head attention	Running several self-attention computations in parallel, each able to specialise differently.
NMS (Non-max suppression)	A post-processing step that removes redundant overlapping detection boxes for the same object.
ONNX	Open Neural Network Exchange; a portable model format enabling deployment across different inference engines.
Panoptic segmentation	Segmentation combining per-pixel class labels with separated object instances.
Precision	Of everything predicted positive, the fraction that was actually positive.
Pruning	Removing weights or channels that contribute little to a model's output, shrinking it.

Term	Definition
Quantisation	Reducing numerical precision of model weights/activations (e.g. FP32 to INT8) to shrink and speed up a model.
Recall	Of everything actually positive, the fraction the model correctly identified.
Receptive field	The region of the input image that influences a given neuron's output.
Reinforcement learning (RL)	Learning a policy via trial-and-error interaction with an environment, guided by a reward signal.
ResNet / residual connection	A 'skip connection' that adds a layer's input to its output, easing training of very deep networks.
Semantic segmentation	Labelling every pixel with a class, without separating individual object instances.
Sim-to-real gap	The performance drop when a model or policy trained in simulation is deployed on real hardware.
SLAM	Simultaneous Localisation and Mapping; estimating a robot's pose while building a map of its environment.
Transfer learning	Reusing a model trained on one (often large) dataset as a starting point for a different, related task.
Transformer	A neural architecture built primarily from self-attention and feed-forward layers, first developed for language.
Vision Transformer (ViT)	A Transformer applied to images by splitting them into patches treated as a token sequence.
VLM (Vision-Language Model)	A model that jointly processes images and text, enabling captioning, visual Q&A, and instruction grounding.
Zero-shot classification	Classifying into categories a model was never explicitly trained on, using aligned embeddings.

Further Reading — Key Papers & Resources

Foundational architectures

- LeCun et al., 'Gradient-Based Learning Applied to Document Recognition' (LeNet, 1998) — the original working convolutional network, applied to digit recognition.
- Krizhevsky, Sutskever & Hinton, 'ImageNet Classification with Deep CNNs' (AlexNet, 2012) — the paper that triggered the deep-learning era in computer vision.
- Simonyan & Zisserman, 'Very Deep Convolutional Networks' (VGG, 2014) — established the value of small, stacked 3×3 filters.
- He et al., 'Deep Residual Learning for Image Recognition' (ResNet, 2015) — introduced skip connections, enabling networks over 100 layers deep.
- Tan & Le, 'EfficientNet: Rethinking Model Scaling for CNNs' (2019) — systematic compound scaling of depth, width, and resolution.

Detection & segmentation

- Ren et al., 'Faster R-CNN: Towards Real-Time Object Detection' (2015) — unified region proposal and detection into one trainable network.
- Redmon et al., 'You Only Look Once: Unified, Real-Time Object Detection' (YOLO, 2016) — the original single-pass detector philosophy.
- He et al., 'Mask R-CNN' (2017) — added a pixel-mask branch to Faster R-CNN via RoIAlign.
- Carion et al., 'End-to-End Object Detection with Transformers' (DETR, 2020) — recast detection as Transformer-based set prediction.
- Kirillov et al., 'Segment Anything' (SAM, 2023) — promptable, open-vocabulary segmentation at scale.

Attention, Transformers & VLMs

- Vaswani et al., 'Attention Is All You Need' (2017) — the original Transformer architecture, introducing self-attention.
- Dosovitskiy et al., 'An Image is Worth 16x16 Words' (ViT, 2020) — applied the Transformer directly to image patches.
- Liu et al., 'Swin Transformer' (2021) — introduced shifted-window local attention for vision.
- Radford et al., 'Learning Transferable Visual Models From Natural Language Supervision' (CLIP, 2021) — contrastive image-text pretraining at scale.
- Liu et al., 'Visual Instruction Tuning' (LLaVA, 2023) — instruction-tuned open vision-language chat.

Robotics, RL & sim-to-real

- Schulman et al., 'Proximal Policy Optimization Algorithms' (PPO, 2017) — a stable, widely used on-policy RL algorithm.
- Haarnoja et al., 'Soft Actor-Critic' (SAC, 2018) — sample-efficient off-policy RL for continuous control.
- Tobin et al., 'Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World' (2017) — the foundational domain-randomisation paper.
- Brohan et al., 'RT-2: Vision-Language-Action Models' (2023) — a single VLM outputting robot actions directly.
- Mur-Artal & Tardos, 'ORB-SLAM: A Versatile and Accurate Monocular SLAM System' (2015) — a widely used feature-based visual SLAM system.

Documentation & tools

- ROS 2 Humble documentation — docs.ros.org — the canonical reference for nodes, topics, services, and TF.
- Gazebo (Harmonic) documentation — gazebosim.org — sensor plugins, SDF world files, and physics configuration.
- Ultralytics YOLO documentation — docs.ultralytics.com — training, export, and deployment guides for the YOLO family.

- PyTorch documentation — pytorch.org/docs — the deep-learning framework used throughout this book's code examples.
- NVIDIA TensorRT documentation — developer.nvidia.com/tensorrt — embedded inference optimisation, referenced in Chapter 13.

Appendix A — Full Mini-Project Code Listing

A consolidated, slightly extended version of the Chapter 12 perception node, including a launch file, ready to drop into a ROS 2 Humble workspace.

A.1 perception_node.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D,
ObjectHypothesisWithPose
from cv_bridge import CvBridge
from ultralytics import YOLO
class PerceptionNode(Node):
def __init__(self):
super().__init__(' cv_bot_perception')
self.declare_parameter(' model_path',
' yolo11n.pt')
self.declare_parameter(' conf_threshold',
0.5)
model_path =
self.get_parameter(' model_path').value
self.conf_th =
self.get_parameter(' conf_threshold').value
self.bridge = CvBridge()
self.model = YOLO(model_path)
self.sub = self.create_subscription(
Image, '/camera/image_raw',
self.on_frame, 10)
self.annotated_pub = self.create_publisher(
Image, '/perception/annotated', 10)
self.detections_pub = self.create_publisher(
Detection2DArray,
'/perception/detections', 10)
self.get_logger().info(' Perception node
ready.')
def on_frame(self, msg: Image):
frame = self.bridge.imgmsg_to_cv2(msg,
' bgr8')
results = self.model(frame, conf=self.conf_th, verbose=False)
det_array = Detection2DArray()
det_array.header = msg.header
for box in results[0].boxes:
det = Detection2D()
hyp = ObjectHypothesisWithPose()
hyp.hypothesis.class_id = str(int(box.cls))
hyp.hypothesis.score = float(box.conf)
```

```
det.results.append(hyp)
x1, y1, x2, y2 = box.xyxy[0].tolist()
det.bbox.center.position.x = (x1 + x2) / 2
det.bbox.center.position.y = (y1 + y2) / 2
det.bbox.size_x = x2 - x1
det.bbox.size_y = y2 - y1
det_array.detections.append(det)
self.detections_pub.publish(det_array)
annotated = results[0].plot()
out = self.bridge.cv2_to_imgmsg(annotated,
' bgr8')
out.header = msg.header
self.annotated_pub.publish(out)
def main(args=None):
rclpy.init(args=args)
node = PerceptionNode()
try:
rclpy.spin(node)
finally:
node.destroy_node()
rclpy.shutdown()
if __name__ == '__main__':
main()
```

A.2 perception.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
return LaunchDescription([
Node(
package=' cv_bot_perception',
executable=' perception_node',
name=' cv_bot_perception',
parameters=[{
' model_path':
' yolo11n.pt',
' conf_threshold': 0.5,
}],
output=' screen',
),
])
```

A.3 package.xml (key dependencies)

```
< depend> rclpy</depend>
< depend> sensor_msgs</depend>
< depend> vision_msgs</depend>
< depend> cv_bridge</depend>
< exec_depend> python3-ultralytics-pip</exec_depend>
```

A.4 Running it

```
colcon build --packages-select cv_bot_perception
```

```
source install/setup.bash
ros2 launch cv_bot_perception perception.launch.py
# In another terminal, visualise:
rviz2 # add Image display -> /perception/annotated
```

Appendix B — Hyperparameter Cheat Sheet

Reasonable starting points for the training runs described throughout this book. Always tune from here based on your own validation curves (Chapter 6), not as fixed rules.

B.1 Image classification (CNN from scratch or fine-tuning)

Hyperparameter	From-scratch (TinyCNN)	Fine-tuning a pretrained backbone
Optimizer	AdamW	AdamW
Learning rate	3e-4	1e-4 (or 1e-5 for the pretrained layers if unfrozen)
Batch size	32–64	16–32
Epochs	30–50	10–20
Weight decay	1e-4	1e-4
LR schedule	Cosine annealing	Cosine annealing with short warm-up

B.2 Object detection (YOLO fine-tuning)

Hyperparameter	Typical value
Image size	640×640 (reduce for embedded deployment, e.g. 416×416)
Batch size	16–32 (GPU-memory dependent)
Epochs	50–100 for a new dataset, 20–50 for fine-tuning an existing checkpoint
Confidence threshold (inference)	0.25–0.5, tuned per Section 4.8
NMS IoU threshold	0.45–0.5

B.3 Reinforcement learning (PPO baseline)

Hyperparameter	Typical value
Learning rate	3e-4
Discount factor (gamma)	0.99
GAE lambda	0.95
Clip range	0.2
Rollout length / batch size	2048–4096 steps per update
Parallel environments	8–32 (as many as compute allows, for sample diversity)

Appendix C — ROS 2 / Gazebo Command Cheat Sheet

C.1 Workspace and build

```
colcon build --packages-select < package_name>
source install/setup.bash
ros2 pkg create --build-type ament_python
< package_name>
```

C.2 Running and inspecting

```
ros2 launch < package_name>
< launch_file>.launch.py
ros2 topic list
ros2 topic echo /camera/image_raw --no-arr
ros2 topic hz /camera/image_raw
ros2 node list
ros2 run rqt_image_view rqt_image_view
```

C.3 TF and debugging

```
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo < source_frame>
< target_frame>
ros2 bag record /camera/image_raw /perception/detections
ros2 bag play < bag_name>
```

C.4 Gazebo

gz sim <world_file>.sdf
ros2 launch ros_gz_sim gz_sim.launch.py gz_args:=<world_file>.sdf
ros2 run ros_gz_sim create -world <world_name> -file <model.sdf> -name <entity_name>

C.5 RViz2

rviz2
rviz2 -d <config_file>.rviz

Appendix D — Troubleshooting Guide

Problem	Likely cause	Fix
No image on /camera/image_raw	Camera plugin missing or misconfigured in SDF/URDF	Check the sensor plugin block and topic remapping; confirm with ros2 topic list
cv_bridge import error	Package not installed, or Python version mismatch	Install ros-humble-cv-bridge; ensure the same Python environment is used throughout
Detections look correct but never reach the controller	Topic name mismatch between publisher and subscriber	Verify exact topic names with ros2 topic list and ros2 topic info

Problem	Likely cause	Fix
Model runs fine on desktop, far too slow on the robot	No hardware-specific optimisation applied	Apply Chapter 13's quantisation and TensorRT export before final deployment
Detections jump around erratically frame to frame	Confidence threshold too low, or no temporal smoothing	Raise the threshold; consider simple smoothing (e.g. a short moving average) across frames
RViz2 shows nothing on the Image display	Wrong topic selected, or QoS mismatch	Re-select the exact topic; check publisher/subscriber QoS reliability settings match
Training loss is NaN within the first few steps	Learning rate too high, or unnormalised inputs	Lower the learning rate by 10×; confirm input normalisation is applied
Simulated robot's camera view looks nothing like expected	Camera link mis-oriented in the URDF/Xacro	Check the camera joint's origin/rotation against the parent link

Appendix E — Common Datasets & Benchmarks

A quick reference to widely used computer vision datasets, useful both for pretraining/benchmarking and for understanding what a pretrained model has (and hasn't) already learned.

Dataset	Domain	Typical use
ImageNet	1.4M general internet images, 1000 classes	Standard classification pretraining/backbone source
COCO	˜330K images, 80 object classes, boxes + masks	Detection and segmentation benchmarking (mAP, mIoU)
Pascal VOC	˜11K images, 20 classes	Smaller, older detection/segmentation benchmark
KITTI	Real-world driving scenes: stereo, LiDAR, odometry	Autonomous driving perception and SLAM benchmarking
nuScenes	1000 driving scenes, full sensor suite	Multi-modal (camera+LiDAR+radar) autonomous driving research
Cityscapes	Urban street scenes, pixel-level semantic labels	Semantic segmentation benchmarking
Open Images	˜9M images, boxes, masks, relationships	Large-scale detection/segmentation pretraining
LAION / WebLI-scale	Billions of web image-text pairs	CLIP-style contrastive pretraining
Objectron / YCB	Household objects with 3D pose annotations	6-DoF pose estimation and grasping research

For your own robotics project, a Gazebo-captured dataset of a few hundred to a few thousand frames — combined with transfer learning from an ImageNet or COCO-pretrained backbone — is almost always sufficient; these large public datasets matter mainly as the source of that pretraining, not as something you need to reproduce yourself.

Appendix F — Sample Project Assessment Rubric

A self-assessment rubric for the Chapter 12 mini-project, useful for tracking your own progress or structuring a course submission.

Criterion	Weight	What 'strong' looks like
Correctness	25%	Perception node reliably publishes accurate detections/masks on live Gazebo camera frames
Understanding	20%	Can explain every stage of the pipeline and justify each model/architecture choice made
Evaluation	15%	Reports mAP/mIoU and latency using the correct metrics from Chapter 6, not just 'it works'
Integration	20%	Perception output demonstrably changes robot behaviour (closed loop), not just visualised in isolation
Engineering quality	10%	Clean ROS 2 package structure, launch files, and parameters (Chapter 12.4)
Documentation	10%	Clear README/report describing architecture, results, and known limitations

Appendix G — Answers & Hints to Review Questions

Brief model answers for self-checking. These are intentionally concise — use them to verify your own, more detailed answer, not as a substitute for working through the question.

Chapter 1

Q#	Answer / hint
1	It keeps the operation local and cheap; local patterns (edges, textures) are what early vision needs, and weight-sharing across positions makes the filter translation-invariant.
2	A layer-1 neuron sees only its filter's small window (e.g. 3×3 pixels); a layer-5 neuron indirectly sees a much larger area because it's built from layer-4 outputs, which were each built from layer-3, and so on.
3	E.g. Convolution (detects local patterns), Pooling (downsamples, adds robustness), Fully Connected (turns features into a final decision).
4	Without non-linearity, stacking any number of linear (convolution) layers collapses mathematically into one single linear layer, losing all the representational benefit of depth.
5	This is overfitting; fix with augmentation/dropout, or with a smaller model/transfer learning from a pretrained backbone.

Q#	Answer / hint
6	The beam width corresponds to the filter (kernel) size — how much local context one filter application considers at once.

Chapter 2

Q#	Answer / hint
1	It solved the vanishing-gradient problem, letting gradients flow through an identity path even in 100+ layer networks that were previously untrainable.
2	Several small 3×3 filters stacked give the same receptive field as one larger filter, but with more non-linearity and fewer parameters.
3	The trade-off between accuracy and compute cost (FLOPs/parameters), scaled jointly across depth, width, and input resolution.
4	EfficientNet or MobileNetV3 — both purpose-built for constrained compute and memory budgets.
5	Reusing pretrained features means only the final task-specific layers need retraining, requiring far less labelled data than training all layers from random weights.
6	It adds a 'direct phone line' (identity path) between detectives, so a message/gradient doesn't degrade after passing through many hands.

Chapter 3

Q#	Answer / hint
1	Cross-entropy treats all misclassifications equally; focal loss down-weights easy/abundant examples so the model focuses on hard, rare ones — useful in detection where background vastly outnumbers objects.
2	SGD+momentum takes a step opposite the gradient with inertia from past steps; Adam adapts a per-parameter learning rate from gradient moments, converging faster but sometimes to a slightly worse final optimum.
3	Geometric: crop, flip, rotation; Photometric: brightness/contrast jitter, colour jitter. Both matter because a robot's camera sees varying viewpoints and lighting.
4	Early stopping protects against overfitting late in training; stop when validation loss stops improving, typically via a patience window.
5	This is overfitting; fix with augmentation/dropout/weight decay, or reduce model capacity, or gather more diverse data.
6	A warm-up phase avoids destabilising large early updates when the model's weights are still poorly initialised for the task.

Chapter 4

Q#	Answer / hint
1	Two-stage first proposes candidate regions then classifies each; one-stage predicts boxes and classes directly in a single pass, trading some accuracy for speed.
2	IoU is the overlap between two boxes (intersection area / union area); NMS uses IoU to decide which overlapping boxes represent the same object so only the highest-confidence one is kept.
3	SSD/MobileNet-SSD is faster and lighter, at the cost of somewhat lower accuracy — a good fit when compute is very constrained.
4	It removes redundant, highly overlapping detections of the same object, keeping only the best one.
5	The spotter is the one-stage detector (YOLO); the specialist assistant + reviewer pair is the two-stage detector (Faster R-CNN).
6	Obstacle avoidance favours recall (catch every possible hazard); grasp targeting favours precision (only act on confident detections).

Chapter 5

Q#	Answer / hint
1	Semantic segmentation labels pixels by class only; instance segmentation additionally separates individual object instances of the same class.
2	RoIAlign avoids the rounding-error misalignment of RoIPooling, giving pixel-accurate feature alignment needed for precise masks.
3	Promptable segmentation accepts a point/box/mask prompt at inference time and generalises to unseen categories, rather than being limited to a fixed trained class list like SAM's predecessors.
4	Segment the object → project the mask onto aligned depth to get a 3D point cloud → fit a grasp candidate → transform into the robot's base frame for the motion planner.
5	Semantic segmentation supports free-space mapping (what's floor vs. wall); instance segmentation supports picking a specific object out of several similar ones.
6	The physiotherapist marking joints corresponds to 6-DoF pose estimation — it captures precise orientation, not just outline.

Chapter 6

Q#	Answer / hint
1	A model that always predicts the majority class can score high accuracy while completely failing on the rare, important class.
2	Precision = TP/(TP+FP) — of predicted positives, how many were correct; Recall = TP/(TP+FN) — of actual positives, how many were found.
3	It rewards models with tighter, more precisely localised boxes by averaging over multiple IoU thresholds rather than accepting any 'roughly correct' overlap.
4	Development workstations (often desktop GPUs) can be 5–10× faster than the embedded GPU actually deployed on the robot.
5	Recall — missing an obstacle is typically far more dangerous than a false alarm.

Q#	Answer / hint
6	Latency, throughput, and memory/power footprint on the actual target hardware.

Chapter 7

Q#	Answer / hint
1	Query/Key/Value are learned projections of each token; attention weight is Query·Key similarity (scaled, softmaxed), used to weight a combination of Value vectors into a new representation.
2	Attention has no innate notion of token order/position, unlike a CNN's implicit spatial structure, so positional information must be added explicitly.
3	It runs several attention computations in parallel, each free to specialise in a different type of relationship, then combines them.
4	Split into patches → flatten & linearly project → add positional embeddings → pass through Transformer encoder blocks → use the [CLS] token's output for classification.
5	Advantage: global context from the first layer. Disadvantage: needs much more training data than a comparable CNN.
6	A high attention weight means the model judges that word highly relevant to interpreting the query word's meaning in context.

Chapter 8

Q#	Answer / hint
1	Contrastive learning pulls matching image-text pairs together and pushes mismatched pairs apart in a shared embedding space, like two librarians filing matching items in the same drawer.
2	Zero-shot classification predicts classes never explicitly trained on, by comparing an image embedding to text embeddings of candidate class descriptions — possible because CLIP aligns vision and language in one space.
3	BLIP-2 adds captioning/Q&A generation; Flamingo interleaves vision features into an LLM via cross-attention for few-shot reasoning; LLaVA projects vision features into an LLM's input space for instruction-following chat.
4	An open-vocabulary detector accepts free-text object descriptions at inference time rather than being limited to a fixed trained class list.
5	It requires reasoning over a disambiguating clause relative to the whole scene, which a simple detector or single-phrase matcher cannot do.
6	Be specific about spatial relationships and distinguishing attributes; request structured output where possible for easier downstream parsing.

Chapter 9

Q#	Answer / hint
1	Task type, latency budget, target hardware, available labelled data, and whether open-vocabulary understanding is needed.
2	Different sub-tasks (fast terrain awareness vs. slower delivery-target recognition) have very different latency/accuracy/vocabulary needs, so one model rarely fits all of them well.
3	Iteration speed, GPU memory during development, dependency footprint, and deployment optimisation effort.
4	Starting simple keeps the system understandable and fast to iterate on, revealing whether extra complexity is actually needed before committing to it.
5	E.g. optimising classification accuracy on a benchmark when the real deployment bottleneck is detection recall on rare safety-critical classes.
6	Transfer learning from a pretrained backbone (Chapter 2) and/or CLIP-style zero-shot approaches (Chapter 8), since both need little or no task-specific labelled data.

Chapter 10

Q#	Answer / hint
1	Sensors → perception → state estimation → policy/planner → actuation, with a feedback loop back to sensors.
2	Camera gives rich semantics but no depth; LiDAR gives precise geometry but no semantics; fusing them covers both gaps.
3	Feature-based SLAM tracks sparse distinctive keypoints frame to frame; direct SLAM uses raw pixel intensities without discrete feature extraction.
4	IMU data fills gaps between slower vision updates and helps reject motion blur, making pose estimates more robust during fast motion.
5	Intrinsic calibration recovers the camera's own optical parameters (focal length, distortion); extrinsic calibration recovers the camera's position/orientation relative to the robot or another sensor. Both are needed for any trustworthy 3D geometry.
6	Fusing frames captured at meaningfully different times silently corrupts the fused estimate, especially during fast motion.

Chapter 11

Q#	Answer / hint
1	It compresses high-dimensional pixel input into a compact latent vector, since learning a policy directly from raw pixels is highly sample-inefficient.
2	RL learns via trial-and-error and a reward signal, needing no expert data; imitation learning is supervised on expert demonstrations, needing no exploration but limited to what was demonstrated.
3	Simulated physics/visuals differ from reality (the 'reality gap'); domain randomisation — randomising textures, lighting, and noise during training — is the standard mitigation.

Q#	Answer / hint
4	E.g. distance to a detected object as a shaping reward, or a penalty when a segmentation mask shows the gripper overlapping an obstacle.
5	The agent finds an unintended way to maximise reward without achieving the real goal; you'd notice by watching recorded rollouts, not just the reward curve.
6	A hybrid approach keeps a well-understood, reliable classical controller for low-level tracking while only the higher-level, harder-to-hand-code decisions are learned.

Chapter 12

Q#	Answer / hint
1	Simulator (Gazebo), middleware (ROS 2), perception node, visualiser (RViz2), and planner/controller node.
2	vision_msgs/Detection2DArray, typically containing per-detection class hypothesis, confidence score, and bounding box.
3	It lets you iterate on the perception node in isolation, much faster than re-running the full Gazebo simulation every time.
4	It generates a diagram of the current TF transform tree, helping spot missing or misconfigured frames.
5	It validates the concepts from Chapters 1–3 end-to-end before relying on a pretrained detector for the rest of the pipeline.
6	Wrapping the loop as a Gymnasium environment and training an RL policy on top of it (Chapter 11).

Chapter 13

Q#	Answer / hint
1	Post-training quantisation quantises an already-trained model directly; quantisation-aware training simulates quantisation effects during training for better accuracy retention.
2	Knowledge distillation trains a small 'student' network to mimic a larger, accurate 'teacher' network's outputs.
3	ONNX is a portable intermediate model format; it lets a PyTorch-trained model run in an optimised inference engine (like TensorRT) without needing PyTorch installed on the robot.
4	Roughly 2–3× latency reduction, per the benchmarking table in Section 13.4.
5	Conversion and quantisation can subtly change numerical behaviour, so pre-quantisation accuracy numbers cannot be assumed to carry over unchanged.
6	A robot processes one live frame at a time in real time, so single-frame latency — not large-batch throughput — is what actually matters.

Chapter 14

Q#	Answer / hint
1	Gazebo Harmonic is the version designed to pair with ROS 2 Humble; Gazebo Classic uses an older, differently structured plugin/SDF API.
2	Isaac Sim offers photorealistic, GPU-accelerated rendering ideal for large-scale RL; Gazebo offers a simpler, more mature, open-source workflow well matched to general robotics development.
3	When a control loop needs low, deterministic latency that Python's overhead cannot reliably guarantee.
4	Foxglove Studio adds richer plotting and log/bag replay in a modern browser-based interface, complementing RViz2's live 3D visualisation.
5	Because the simulator already knows exact ground-truth labels (poses, masks), so large labelled datasets can be generated automatically, far faster than manual annotation.
6	Gazebo Harmonic for simulation, a Python/rclpy perception node, RViz2 for visualisation, and the existing MPC/planner for control, with TensorRT export before real-hardware deployment.

Appendix H — Extended FAQ

Do I need a GPU to work through this book?

For the small examples (TinyCNN, a handful of epochs) a CPU is fine, if slow. For fine-tuning YOLO or a ViT on more than a few hundred images, a GPU (even a modest laptop GPU, or a free cloud notebook GPU) will save hours.

Should I learn PyTorch or TensorFlow first?

This book uses PyTorch throughout because it dominates current robotics and research tooling (including Ultralytics YOLO). TensorFlow remains capable, but PyTorch is the more useful default for the ROS 2 + Gazebo path this book follows.

Do I need to understand the maths of backpropagation in detail?

A working intuition (gradients flow backward, telling each weight how to change to reduce the loss) is enough to use these tools well. A full derivation is valuable for research but not required to build the mini-project in Chapter 12.

Is ROS 1 still worth learning?

ROS 2 (this book's focus) is the actively developed, production-supported line; ROS 1 reached end-of-life. Unless you are maintaining an existing ROS 1 codebase, start directly with ROS 2 Humble.

My model works in Gazebo but fails on a real camera — why?

This is the sim-to-real gap (Chapter 11). Check lighting/texture differences first, apply domain randomisation during simulated training, and fine-tune on a small amount of real captured data if available.

How much labelled data do I actually need?

For fine-tuning a pretrained detector on a narrow task, a few hundred well-varied images is often enough to get a usable result; a few thousand gives meaningfully more robustness. Quality and

variety (angles, lighting, backgrounds) matter more than raw count.

Can I skip straight to the robotics chapters (10–14)?

You can, but you will move faster with at least Chapters 1, 2, 4, and 7–8 read first — the robotics chapters assume familiarity with CNNs, detection, and VLM grounding.

What's the single highest-leverage thing to learn first?

Get a pretrained YOLO model running end-to-end on your own images (Chapter 4), even before deeply understanding the architecture — seeing the full pipeline work builds intuition that makes the theory chapters click faster.

Why does the book recommend starting with a small model rather than the largest available?

Faster iteration and full understanding of every stage matter more early on than squeezing out the last few points of accuracy — see Chapter 9's cost-of-ownership discussion.

Is Gazebo Classic the same as Gazebo (used in this book)?

No — Gazebo Classic is the older, separately maintained line. This book uses Gazebo Harmonic, the version designed to pair with ROS 2 Humble; APIs and SDF syntax differ between the two.

How do I know if my perception pipeline is 'good enough' to stop optimising?

Compare your measured latency (Chapter 6) against your control loop's required rate, and your mAP/mIoU against your task's error tolerance. Once both comfortably clear their targets, further optimisation has diminishing returns relative to the project's other risks.

Do VLMs replace the need for traditional detectors on a robot?

Not for the real-time control loop — VLMs are typically far too slow and expensive to run every frame. They are best used for occasional, higher-level reasoning alongside a fast dedicated detector, not as a full replacement.

What's a reasonable first hardware target if I don't yet have a robot?

Prototype entirely in Gazebo first (Chapter 12) — a simulated camera, LiDAR, and robot base are enough to build and validate the full perception pipeline before any hardware purchase is needed.

How do I decide between fine-tuning a pretrained detector versus training one from scratch?

Default to fine-tuning (Chapter 2.3) unless your objects are visually very unlike anything in typical pretraining data (e.g. unusual sensor modalities), or you specifically want the learning experience of training from scratch (Chapter 1.9).

Is it worth learning C++ for ROS 2 if I'm comfortable in Python?

For perception and prototyping, Python is usually sufficient and faster to iterate in. Learn C++ specifically when you hit a node that needs guaranteed low, deterministic latency that Python's overhead can't reliably provide (Chapter 14.2).

My mAP looks good but the robot still behaves badly — why?

mAP measures detection quality in isolation; it doesn't capture calibration errors, timing/synchronisation issues, or how the planner consumes detections. Check Chapter 10's calibration and sensor-fusion sections, and verify the closed-loop behaviour directly, not just the offline metric.

How often should I retrain or re-evaluate a deployed model?

Whenever the deployment environment changes meaningfully (new lighting, new object variants, a hardware revision), and periodically even without a known change, to catch silent drift — tracking model versions as described in Chapter 13.8 makes this routine rather than risky.

Appendix I — Common ROS 2 Message Types Reference

A quick reference to the message types most relevant to a computer-vision robotics pipeline, as used throughout Part IV.

Message type	Package	Typical use
Image	sensor_msgs	Raw or annotated camera frames
CameraInfo	sensor_msgs	Intrinsic calibration parameters published alongside a camera stream
PointCloud2	sensor_msgs	3D point data from depth cameras or LiDAR
Imu	sensor_msgs	Orientation, angular velocity, and linear acceleration from an IMU
LaserScan	sensor_msgs	2D range data from a planar LiDAR
Detection2DArray / Detection2D	vision_msgs	A set of 2D detections, each with class hypothesis, score, and bounding box
Detection3DArray / Detection3D	vision_msgs	3D detections, used for pose-aware perception outputs
Twist	geometry_msgs	Linear and angular velocity commands, e.g. published to cmd_vel
PoseStamped	geometry_msgs	A timestamped 3D position and orientation, e.g. a grasp target pose
TFMessage	tf2_msgs	Transform tree updates between coordinate frames
OccupancyGrid	nav_msgs	A 2D probabilistic map of free/occupied space
Odometry	nav_msgs	Estimated robot pose and velocity over time

Appendix J — Robotics Acronym Reference

Acronym	Meaning
AMR	Autonomous Mobile Robot
DOF	Degrees of Freedom
EKF	Extended Kalman Filter
FPS	Frames Per Second

Acronym	Meaning
IMU	Inertial Measurement Unit
mAP	mean Average Precision
mIoU	mean Intersection over Union
MPC	Model Predictive Control
NMS	Non-Max Suppression
ONNX	Open Neural Network Exchange
PPO	Proximal Policy Optimisation
RL	Reinforcement Learning
ROS	Robot Operating System
RViz	ROS Visualisation (tool)
SAC	Soft Actor-Critic
SDF	Simulation Description Format
SLAM	Simultaneous Localisation and Mapping
TF	Transform (coordinate frame tree in ROS)
URDF	Unified Robot Description Format
VIO	Visual-Inertial Odometry
ViT	Vision Transformer
VLM	Vision-Language Model

Appendix K — Practice Problem Set

Seventeen worked problems spanning the whole book, useful as a self-test before attempting the Chapter 12 mini-project.

Problem 1

A 224×224 RGB image passes through two consecutive 3×3 convolution layers, each with stride 1 and 'same' padding, followed by a 2×2 max-pool with stride 2. What is the spatial size of the output feature map?

Solution. Both 3×3 'same'-padded convolutions preserve spatial size (224×224 after each). The 2×2/stride-2 max-pool then halves it: 224/2 = 112. Output: 112×112 (channel count depends on the number of filters, not asked here).

Problem 2

A predicted bounding box is (x1=20, y1=20, x2=120, y2=120). The ground-truth box is (x1=60, y1=60, x2=160, y2=160). Compute the IoU.

Solution. Intersection: x from 60–120, y from 60–120 → 60×60 = 3,600. Each box area = 100×100 = 10,000. Union = 10,000+10,000-3,600 = 16,400. IoU = 3,600/16,400 ≈ 0.220.

Problem 3

A detector reports 8 true positives, 2 false positives, and 3 false negatives on a validation set. Compute precision and recall.

Solution. Precision = TP/(TP+FP) = 8/10 = 0.80. Recall = TP/(TP+FN) = 8/11 ≈ 0.727.

Problem 4

A model runs at 45 ms latency per frame on a desktop GPU and 210 ms on a Jetson-class board. What is the maximum sustainable FPS on each, and would either meet a 20 Hz control loop requirement?

Solution. Desktop: $1000/45 \approx 22.2$ FPS — meets a 20 Hz requirement. Jetson: $1000/210 \approx 4.8$ FPS — does not meet it; would need optimisation (Chapter 13) such as TensorRT FP16/INT8 export.

Problem 5

A self-attention layer processes a sequence of 196 patch tokens (from a 14×14 grid of ViT patches). Approximately how does the attention computation's cost scale if the number of patches doubles to 392?

Solution. Self-attention cost scales quadratically with the number of tokens (each token attends to every other token), so doubling the tokens roughly quadruples the attention computation cost ($2^2 = 4\times$), not merely doubling it.

Problem 6

A CLIP model computes cosine similarity of 0.31 between an image embedding and the text 'a robot arm', and 0.09 with 'an empty table'. After softmax over just these two candidates (temperature-scaled similarities of 3.1 and 0.9), which class is predicted and roughly how confident is the model?

Solution. The much higher similarity (3.1 vs 0.9 after scaling) means softmax will assign the large majority of probability mass to 'a robot arm' — e^3.1/(e^3.1+e^0.9) ≈ 0.90, so roughly 90% confidence in 'a robot arm'.

Problem 7

A PyTorch FP32 model is 84 MB. Estimate its approximate size after FP16 quantisation, and after INT8 quantisation.

Solution. FP16 halves storage per parameter (32-bit → 16-bit): roughly 42 MB. INT8 quarters the original (32-bit → 8-bit): roughly 21 MB (actual sizes vary slightly due to non-quantised layers and metadata).

Problem 8

A robot's camera publishes at 30 Hz, but the perception node's inference takes 60 ms per frame. What happens if no frame-dropping logic is used, and what is one practical fix?

Solution. 60 ms per frame limits processing to ˜16.7 FPS, so frames will queue up faster than they're processed, causing growing latency ('lag') over time. A practical fix is to process only the most recent frame (drop stale queued frames) rather than processing every frame in order.

Problem 9

An RL policy trained purely in simulation achieves a 95% task success rate in Gazebo but only 40% on real hardware. Name the phenomenon and two concrete mitigations.

Solution. This is the sim-to-real gap. Mitigations: domain randomisation during simulated training (Chapter 11), and fine-tuning or calibrating with a small amount of real-world data/observations.

Problem 10

A dataset has 300 labelled images. Using 5-fold cross-validation, how many images are used for validation in each fold, and how many total training runs are performed?

Solution. Each fold holds out $300/5 = 60$ images for validation (with the remaining 240 used for training in that fold); 5 total training runs are performed, one per fold.

Problem 11

A segmentation model achieves a Dice coefficient of 0.85 on a validation mask. Is this generally considered strong, moderate, or weak performance, and what related metric would you also want to check?

Solution. 0.85 is generally considered strong performance for most segmentation tasks (values closer to 1.0 are better). It's still worth checking mIoU and per-class performance, since a high aggregate Dice score can still hide poor performance on a rare or small class.

Problem 12

A robot's obstacle detector has 98% recall and 60% precision. Its grasp-target detector has 70% recall and 97% precision. Explain why these different operating points make sense given each detector's role.

Solution. The obstacle detector prioritises catching every possible hazard (high recall) even at the cost of some false alarms (lower precision), since missing an obstacle is dangerous. The grasp-target detector prioritises only acting on confident detections (high precision) since a wrong grasp attempt is costly, even if it means occasionally missing a valid target (lower recall).

Problem 13

A ViT-B/16 model processes a 224×224 image. How many patch tokens does it produce (excluding the [CLS] token)?

Solution. $224/16 = 14$ patches per side, so $14 \times 14 = 196$ patch tokens, plus the [CLS] token bringing the total sequence length to 197.

Problem 14

An RL training run uses 16 parallel simulated environments, each collecting 128 steps before a policy update. How many environment steps does one update use in total, and roughly how many updates are needed to reach 1,000,000 total steps?

Solution. $16 \times 128 = 2{,}048$ steps per update. $1{,}000{,}000 / 2{,}048 \approx 488$ updates.

Problem 15

A colcon workspace fails to build with an error referencing a missing 'vision_msgs' dependency. What are two likely causes and fixes?

Solution. Either the package.xml is missing the <depend>vision_msgs</depend> declaration (fix: add it and rebuild), or the vision_msgs ROS 2 package itself isn't installed on the system (fix: sudo apt install ros-humble-vision-msgs or equivalent for your distribution).

Problem 16

A robot's perception node subscribes to /camera/image_raw but 'ros2 topic hz' shows 0 Hz. List three things to check, in the order you'd check them.

Solution. 1) Is the camera/Gazebo sensor plugin actually running and spawned correctly (check 'ros2 topic list' for the topic's existence at all). 2) Is the topic name an exact match, including any namespace prefix (a common typo source). 3) Is there a QoS mismatch between publisher and subscriber (e.g. one using best-effort, the other reliable) silently preventing delivery.

Appendix L — Capstone Case Study: Warehouse Pick-and-Place Robot

This appendix walks through one larger, realistic scenario end to end, showing how every part of this book fits together in a single system — useful as a template for scoping your own project.

L.1 Scenario

A wheeled mobile manipulator operates in a small simulated warehouse aisle (Gazebo Harmonic). Its task: navigate to a shelf, identify a specific labelled box among several similar ones, pick it up, and deliver it to a drop-off zone, while avoiding other moving obstacles.

L.2 System breakdown by book chapter

Sub-system	Approach used	Relevant chapters
Navigation to the shelf	Visual-inertial SLAM for localisation; classical MPC trajectory tracking	3, 10, 14
Obstacle avoidance en route	Real-time YOLO-nano detection fused with LiDAR	4, 6, 10
Identifying the correct box	Open-vocabulary detection (label text match) or a fine-tuned detector on known box classes	2, 4, 8, 9
Precise grasp planning	Instance segmentation → depth projection → 6-DoF pose estimation	5
Manipulation execution	Classical motion planning given the estimated grasp pose (outside CV scope, but consumes its output)	5, 10
Handling ambiguous instructions	VLM grounding for cases like 'the box on the left, not the one behind it'	8
Onboard deployment	TensorRT-optimised detector and segmentation model on a Jetson-class computer	13

L.3 Design decisions and their trade-offs

- **Two detectors, two thresholds** — the obstacle-avoidance path favours recall (Chapter 4.8); the box-identification path favours precision, since misidentifying the box is costly.
- **Segmentation only where needed** — full instance segmentation only runs once the robot is within grasping range of the shelf, not continuously, to save onboard compute (Chapter 5.7).
- **VLM as a fallback, not the default path** — the fast fine-tuned detector handles the common case; the slower VLM is only invoked when the instruction contains disambiguating

language the detector's fixed class list can't resolve (Chapter 8.7).

L.4 What could go wrong, and how each chapter helps

Failure mode	Where to look
Robot picks the wrong box	Re-check detector precision/threshold (Ch. 4, 6); consider adding VLM disambiguation (Ch. 8)
Grasp misses or drops the box	Re-check camera-depth calibration (Ch. 10.6) and grasp-candidate scoring (Ch. 5.5)
Works in Gazebo, fails on real hardware	Sim-to-real gap; apply domain randomisation and real-data fine-tuning (Ch. 11)
Perception too slow for the control loop	Profile and optimise per Chapter 13; consider a lighter backbone (Ch. 2)
Robot collides with a moving obstacle	Check obstacle detector recall and sensor fusion synchronisation (Ch. 10.7)

This scenario intentionally mirrors the structure of the Chapter 12 mini-project, scaled up: the same architecture pattern — fast dedicated models for the real-time loop, heavier models invoked selectively, and everything measured against the metrics in Chapter 6 — generalises well beyond this specific warehouse example to most robot perception systems.

L.5 Sketch of the top-level state machine

The scenario's behaviour is naturally expressed as a small state machine, with vision-derived signals driving each transition:

```
NAVIGATE_TO_SHELF
-> on arrival (SLAM pose within tolerance): IDENTIFY_BOX
IDENTIFY_BOX
-> on confident match (detector/VLM score > threshold): APPROACH_GRASP
-> on ambiguous match: REQUEST_VLM_DISAMBIGUATION -> IDENTIFY_BOX
APPROACH_GRASP
-> on grasp pose computed (Ch. 5.5 pipeline): EXECUTE_GRASP
EXECUTE_GRASP
-> on grasp success (gripper feedback + visual re-check): NAVIGATE_TO_DROPOFF
-> on grasp failure: APPROACH_GRASP (retry, up to N times)
NAVIGATE_TO_DROPOFF
-> on arrival: RELEASE_BOX -> IDLE

# Running in parallel throughout: OBSTACLE_MONITOR
# (Ch. 4/10 detector+LiDAR fusion) can pre-empt any state
# with an EVASIVE_STOP if a collision risk is detected.
```

Each state's entry/exit condition is exactly one of this book's earlier outputs — a SLAM pose, a detector confidence score, a computed grasp pose, or a fused obstacle signal — which is the general pattern for wiring perception into robot behaviour: perception modules produce well-defined signals, and a simple, inspectable state machine (or behaviour tree) decides what to do with them, rather than perception and decision-making being tangled into one monolithic piece of code.

Appendix M — Suggested 8-Week Study & Project Plan

A pacing guide for working through this book alongside the Chapter 12 mini-project, assuming a few focused hours per week. Adjust freely — the dependencies matter more than the exact week numbers.

Week	Focus	Deliverable
1	Chapters 1–2 (CNN foundations, architecture evolution)	TinyCNN trained and validated on a small dataset
2	Chapter 3 (training) + Gazebo/ROS 2 environment setup	Robot spawns in Gazebo with a working camera topic
3	Chapter 4 (detection)	Pretrained YOLO running inference on Gazebo camera frames
4	Chapters 5–6 (segmentation, pose, evaluation)	mAP/mIoU measured on your own validation split
5	Chapters 7–8 (Transformers, VLMs)	CLIP zero-shot classification demo on captured frames
6	Chapter 9 + Chapter 12 (project build)	Perception node integrated with RViz2 visualisation
7	Chapters 10–11 (robotics integration, RL) + close the loop	Robot behaviour visibly reacts to detections
8	Chapter 13–14 (deployment) + polish	Benchmarked, TensorRT-exported model; final report/demo

Index

Term, followed by the section number(s) where it is introduced or discussed in depth.

Term	Term
6-DoF pose, 5.4, Glossary	Accuracy, 6.1
Adam / AdamW, 3.2, Appendix B	AlexNet, 2.1
Anchor box, 4.2, Glossary	Attention, 7.1–7.2, Glossary
Augmentation, 3.3	Backbone, 2.3, Glossary
Batch normalisation, 1.5, Glossary	CLIP, 8.1, Glossary
Confusion matrix, 6.2	Contrastive learning, 8.1, Glossary
Convolution, 1.2, Glossary	DETR, 4.1
Domain randomisation, 11.4, Glossary	EfficientNet, 2.1
Focal loss, 3.1	Fine-tuning, 2.3, Glossary
GoogLeNet / Inception, 2.1	Imitation learning, 11.2, Glossary
Inductive bias, 7.4, Glossary	IoU, 4.2, 6.3, Glossary
Jetson (edge deployment), 6.5, 13.4	LeNet-5, 2.1
mAP, 6.3, Glossary	Mask R-CNN, 5.2
MobileNet, 2.1	NMS, 4.2, Glossary
ONNX, 13.3, Glossary	Panoptic segmentation, 5.1
Positional encoding, 7.7	Precision / Recall, 6.1
Pruning, 13.2	Quantisation, 13.1, Glossary

Term	Term
Receptive field, 1.7, Glossary	ResNet, 2.1, Glossary
Reward shaping, 11.5	RViz2, 12.1, 14.3
SAC / PPO, 11.3, Appendix B	SAM (Segment Anything), 5.3
SLAM, 10.3, Glossary	Swin Transformer, 7.5
TensorRT, 13.3	Transfer learning, 2.3, Glossary
VGGNet, 2.1	ViT (Vision Transformer), 7.3, Glossary
VLM, 8.2, Glossary	YOLO, 4.3